



NUS Orbital 2026

[Apollo]

Joshua Andrew - A0308971W

Huang Sijia - A0320782H

RAGsToRiches

Smarter resumes. Smarter opportunities.

AI feedback grounded in the Google XYZ formula, the STAR method, and the Rule of Three; free, transparent, and without black boxes.

+ Privacy forward + Zero paywalls + Evidence-based

An open-source, Retrieval-Augmented Generation (RAG) powered resume analysis platform designed to provide transparent, evidence-based, and accessible resume coaching. Built *by students, for students*, RAGsToRiches combines modern Large Language Models (LLMs), semantic retrieval, and document analysis into a modular pipeline that helps candidates improve their resumes without sacrificing factual accuracy or personal ownership.

Developed as an independent project for **NUS Orbital 2026 (Apollo 11)**.

Table of Contents

0. Executive summary

1. Introduction
2. Motivation
3. Design Philosophy
4. Project Scope
5. System Overview
6. High-Level System Architecture
7. Technical Implementation
8. UI/UX Design
9. Repository Structure
10. Deployment
11. Design Rationale
12. Engineering Challenges
13. Testing & Validation
14. Current Features
15. Future Work
16. Lessons Learned
17. Contributors
18. References
19. Final Remarks

0 Executive Summary

0.1 Project Overview

RAGsToRiches is an open-source, Retrieval-Augmented Generation (RAG) powered resume analysis platform developed as part of **NUS Orbital 2026 (Apollo)**.

The project aims to make professional-quality resume feedback freely accessible by combining modern Large Language Models (LLMs) with evidence-based resume writing frameworks. Instead of producing generic AI suggestions, every significant recommendation is grounded in retrieved knowledge from a curated vector database containing recognised industry methodologies such as the Google XYZ Formula, the STAR Method, the Rule of Three, and Applicant Tracking System (ATS) best practices.

Users register an account, upload a resume, optionally provide a job description, and receive a comprehensive analysis consisting of resume scoring, ATS feedback, framework-grounded bullet rewrites, document generation, and personalised recommendations. Throughout the process, users remain in complete control over every modification through an interactive review workflow that requires explicit acceptance before any rewrite is incorporated into the final document.

As of Milestone 3, the platform has evolved into a full multi-user web application supporting two roles: candidates (job seekers who upload and iterate on their resumes) and mentors (career coaches or university staff who review candidates' work and provide structured feedback). The application is now served as a React single-page application backed by an Express REST API and a private python analysis engine, replacing the earlier Streamlit prototype.

Unlike many commercial resume optimisation platforms, RAGsToRiches prioritises explainability, factual accuracy, transparency and privacy over automated generation alone. The system is capable of operating with both cloud-hosted models and locally hosted language models through Ollama, allowing privacy-conscious users to analyse sensitive career documents entirely offline.

Ultimately, the project seeks to demonstrate that AI-assisted career guidance can be both technically sophisticated and fundamentally trustworthy.

0.2 Motivation

Artificial intelligence has dramatically lowered the barrier to producing written content. Resume writing has been no exception. Numerous commercial platforms now promise instant resume optimisation, ATS compatibility, and personalised career advice using large language models.

However, these platforms frequently suffer from several fundamental limitations.

Most AI resume tools today fall into one of two traps: they are locked behind expensive paywalls, or they offer generic, black-box advice that cannot be tailored to the specific roles a candidate is targeting. Worse, many LLM-based tools suffer from hallucinations — fabricating metrics, suggesting irrelevant keywords, or rewriting bullet points with made-up achievements that undermine the candidate's credibility.

While such changes may appear persuasive, they ultimately undermine the credibility of the applicant and expose them to significant professional risk.

There is a clear gap in the market for a tool that is both free and grounded. RAGsToRiches addresses this by leveraging Retrieval-Augmented Generation (RAG) to anchor every suggestion in established, evidence-based resume frameworks, specifically the Google XYZ formula ("Accomplished X, as measured by Y, by doing Z"), the Rule of Three, and Applicant Tracking System (ATS) best practices. Rather than hallucinating improvements, the system grounds every rewrite within a curated repository of established resume-writing frameworks retrieved dynamically through Retrieval-Augmented Generation, then applies it to the candidate's actual content. Every piece of feedback is traceable to a known rubric, making the advice transparent, defensible, and genuinely useful.

Explicit placeholders are used whenever measurable achievements are unavailable, ensuring that fabricated statistics are never introduced into a user's resume.

Beyond producing stronger resumes, the project also aims to educate its users. Every recommendation is accompanied by the framework that informed it, allowing candidates to understand *why* a suggestion was made rather than blindly accepting AI-generated content.

The long-term objective is simple: to provide students and job seekers with free, transparent, evidence-based resume guidance that has traditionally been available only through expensive career coaching services.

0.3 Aim

The primary objective of RAGsToRiches is to develop an end-to-end resume analysis platform that combines modern language models with Retrieval-Augmented Generation to deliver trustworthy, explainable, and actionable resume feedback.

Starting from a resume uploaded by the user, the system performs document parsing, structural analysis, semantic evaluation, framework retrieval, ATS comparison, and targeted bullet-point rewriting before generating revised application documents.

Unlike conventional AI writing assistants, the project deliberately avoids acting as an autonomous editor. Instead, it functions as an intelligent decision-support system that assists users in improving their resumes while preserving their ownership over every modification.

The project also demonstrates that modern LLM applications can be designed around engineering principles such as modularity, provider independence, transparency, and factual grounding without sacrificing usability or analytical capability.

0.4 Vision

The long-term vision of RAGsToRiches extends beyond merely producing stronger resumes.

RAGsToRiches will be a **free, intelligent resume improvement platform** that empowers students and job seekers to present their experience with clarity, precision, and impact, without relying on expensive career coaches or opaque AI black boxes.

RAGsToRiches will ground every suggestion in established, **evidence-based**, and industry-recognized frameworks, retrieved in real time from a curated vector knowledge base. Rather than generating generic rewrites, the system will **explain the reasoning behind every change**, helping users not just improve their current resume but **develop a deeper understanding** of what strong, results-oriented writing looks like.

By providing a **quantitative resume score** broken down by **measurable** dimensions: achievement quantification, action verb strength, and structural clarity, RAGsToRiches will give users a concrete baseline to work from and a clear sense of progress as they iterate. The side-by-side bullet rewriter will make improvements **immediately visible**, putting users in control of every change rather than blindly accepting AI output.

For users targeting specific roles, RAGsToRiches will offer **ATS keyword matching** against any job description, ensuring that resumes are not just well-written in the abstract but **strategically aligned to the requirements of the exact position** being applied for.

Ultimately, RAGsToRiches aims to level the playing field, giving every student and career-switcher access to the kind of **structured, personalised** resume feedback that has traditionally been available only to those who can afford professional coaching.

As the platform continues to evolve, future iterations will support larger knowledge bases, collaborative editing, recruiter-facing workflows, custom fine-tuned resume models, and deeper integration with recruitment ecosystems while maintaining the project's commitment to openness, transparency, and user control.

0.5 Project Scope

RAGsToRiches is intentionally scoped as an **end-to-end** resume analysis and application optimisation platform. A typical workflow consists of creating a user account, uploading a resume in one of several supported document formats, optionally supplying or saving a target job description, receiving a comprehensive AI-assisted analysis, reviewing framework-grounded recommendations, accepting or rejecting suggested rewrites, and exporting tailored application documents for submission.

Unlike the earlier prototype, which focused on a single-session analysis workflow, the current platform supports persistent multi-user accounts, secure authentication, historical analysis records, mentor review workflows, and personalised configuration through user-managed API keys. Candidates may iteratively refine multiple resumes over time, while mentors are able to review previous submissions, compare revisions, and provide structured feedback through a dedicated review interface.

The project deliberately focuses on improving resume quality and application readiness rather than replacing the wider recruitment ecosystem. Although users may tailor resumes against individual job descriptions, generate targeted CVs and cover letters, and import professional profile information from supported sources such as LinkedIn exports, the platform does not function as a job board, applicant tracking system, or recruitment marketplace. Instead, it is designed to complement existing recruitment platforms by providing evidence-based resume analysis before applications are submitted.

This focused scope allows the project to prioritise analytical quality, explainability, factual grounding, privacy, and reliability within a clearly defined problem domain, while simultaneously supporting a substantially broader end-to-end workflow than the original prototype.

0.6 Why RAGsToRiches?

The AI-assisted resume-writing landscape has grown rapidly over the past few years. Most existing AI resume tools fall into one of two categories: free tools that give generic, unverifiable advice ("use more action verbs!"), or paid tools that lock meaningful feedback behind a subscription. RAGsToRiches is built around a different premise: that AI feedback is only useful if the person receiving it can trust **why** a suggestion was made.

Rather than attempting to maximise the quantity of AI-generated output, the project focuses on producing recommendations that are explainable, evidence-based, and ultimately controlled by the user. Several design decisions distinguish the system from conventional AI resume optimisation platforms.

Framework-Grounded Rather Than Prompt-Grounded

Most resume-writing assistants rely almost entirely on prompt engineering. The language model is instructed to "improve" a bullet point, often without any external knowledge or established writing standard.

RAGsToRiches instead adopts a Retrieval-Augmented Generation (RAG) architecture. Before any rewrite is produced, the system retrieves the most relevant resume-writing frameworks from a curated ChromaDB knowledge base containing resources such as the Google XYZ Formula, STAR Method, Rule of Three, and ATS action verb libraries.

These retrieved documents are injected directly into the prompt, ensuring that recommendations are grounded in recognised industry practices rather than generated solely through model intuition. Every rewrite is accompanied by the framework that informed it, allowing users to understand both *what* changed and *why* it changed.

Human-in-the-Loop Decision Making

Many commercial resume tools immediately replace existing content with AI-generated alternatives, encouraging users to accept suggestions without careful evaluation.

RAGsToRiches deliberately avoids this workflow.

Every suggested rewrite is presented individually alongside the original bullet point, together with the framework used and a concise explanation describing the rationale behind the recommendation. Users explicitly decide whether to accept or reject each suggestion before it contributes to the final generated resume.

This review process positions the application as an intelligent writing assistant rather than an autonomous editor, ensuring that ownership of the resume always remains with the candidate.

Transparent and Explainable Analysis

Transparency represents one of the project's defining principles.

Rather than presenting only a final resume score, the application exposes how that score was derived. Users can inspect individual scoring components, including structural completeness, bullet quality, action verb usage, keyword coverage, and identified weaknesses.

Similarly, recommendations throughout the application are accompanied by explanatory text that describes the reasoning behind each suggestion instead of simply presenting revised content.

The objective is not merely to improve a single resume, but to help users understand the underlying principles of effective resume writing so that they can apply those techniques independently in future applications.

Factual Grounding Over Hallucination

Large language models are capable of producing convincing yet inaccurate statements, particularly when attempting to strengthen resume bullet points.

To minimise this risk, RAGsToRiches adopts a deliberately conservative generation strategy.

The system never fabricates measurable achievements. Whenever quantitative evidence is unavailable, placeholders such as **[X%]**, **[N users]**, or **[Revenue]** are inserted instead, explicitly prompting users to replace them with genuine values obtained from their own work.

Additionally, an optional agentic critic stage performs a secondary review of generated rewrites to identify fabricated metrics, unsupported claims, or inconsistencies with the original resume before recommendations are presented to the user.

This emphasis on factual accuracy prioritises professional integrity over superficially impressive AI output.

Privacy-Forward and Provider Agnostic

Resume documents frequently contain sensitive personal information, including employment history, educational records, contact information, and project experience.

Recognising these privacy concerns, the system is intentionally designed to support multiple deployment models.

Users may analyse resumes using commercial cloud providers such as Google Gemini, Anthropic Claude, and OpenAI, or execute the complete pipeline locally using Ollama-hosted language models without transmitting resume data to external services.

A provider-agnostic routing layer abstracts interactions with every supported model, allowing users to select the provider that best matches their requirements for cost, privacy, reasoning quality, or latency without requiring modifications elsewhere in the application.

Free and Accessible by Design

Professional resume coaching and AI-powered career tools frequently require recurring subscription fees, limiting access for the very audience that benefits most from them.

RAGsToRiches was developed with a different objective.

The project is open-source and designed to provide students, graduates, and job seekers with access to high-quality, evidence-based resume analysis without placing core functionality behind premium tiers. Features such as framework-grounded rewriting, resume scoring, ATS analysis, document generation, and local model execution remain fundamental capabilities of the platform rather than paid upgrades. There is no paywall gating the rewrite engine, the keyword matcher, or the CV/cover letter generation. The project's premise is that high-quality, evidence-based resume feedback shouldn't be reserved for users who can pay for it.

Together, these principles define the identity of RAGsToRiches. Every major architectural decision, from parser design and Retrieval-Augmented Generation to the multi-provider routing layer, interactive review interface, and deployment strategy, was shaped by a single objective: to build an AI resume assistant that users can understand, trust, and remain in control of throughout the entire analysis process.

0.7 Project Features & User Stories

Feature	User Stories
PDF-to-Text Parser	As a student applicant, I want to upload my resume as a PDF(or other formats) so that I can receive feedback without having to manually copy and paste my content into the system.
	As a user with a non-standard resume layout (e.g. two-column format or creative template or other formats), I want the parser to correctly extract all my content so that no experience or skill is accidentally omitted from the analysis.
Vector Knowledge Base	As a user, I want the AI's suggestions to be grounded in established resume frameworks, so that I can trust the feedback is based on real industry standards rather than generic AI output.
	As a user, I want to know which specific framework or guideline informed each suggestion, so that I can learn the reasoning behind the improvement and apply it independently in the future.
Resume Score & Impact Analyzer	As a student applicant, I want to receive an overall resume score so that I can quickly understand how competitive my resume is before sending it to employers.
	As a user, I want the score to be broken down by category so that I know exactly which area to prioritise when making improvements.
	As a career-switcher, I want the analyzer to flag bullet points that lack measurable outcomes so that I can identify which parts of my experience are not communicating value effectively to recruiters.
Side-by-Side Bullet Rewriter	As a user, I want to see my original bullet point displayed next to the AI-suggested rewrite so that I can clearly understand what changed and why.
	As a user, I want to be able to accept, reject, or manually edit each suggested rewrite so that I remain in full control of my resume content and voice.
	As a user who is unfamiliar with the Google XYZ formula, I want each rewrite to be accompanied by a brief explanation of the framework it applies so that I can learn to write stronger bullet points on my own going forward.
	As a career-switcher, I want the system to compare my resume against a specific job description so that I can identify missing keywords that

ATS Keyword Matcher	Applicant Tracking Systems would flag before my resume reaches a human recruiter.
	As a user applying to multiple roles, I want to paste a different job description each time so that I can tailor my resume to each specific role without needing a separate tool.
Agentic Self-Correction	As a user, I want the AI to verify its own suggestions before presenting them to me so that I can trust that fabricated achievements have been identified and removed. As a developer, I want to benchmark the critic pipeline against the baseline generator so that I can objectively evaluate the benefits and trade-offs of agentic self-correction..
User Analytics Dashboard	As a candidate using the tool over multiple weeks, I want to see a chart of my resume score over time so that I can measure whether my revisions are making a meaningful difference. As a user, I want a readability heatmap showing which sections of my resume are strongest and weakest so that I know where to focus my editing effort. As a user, I want previous cover letters and generated resumes to remain accessible so that I can reuse or adapt them for future job applications. As a mentor, I want to review a candidate's historical submissions so that I can understand their long-term development rather than assessing a single snapshot.
Mentor Review Workspace	As a mentor, I want to compare two resume versions side-by-side so that I can immediately identify changes between revisions. As a mentor, I want to annotate specific suggestions and provide structured feedback so that candidates understand the reasoning behind my recommendations. As a candidate, I want my mentor's comments to be integrated into the same review workflow as the AI suggestions so that all feedback remains organised in one place.
Job Description & LinkedIn Integration	As a job seeker, I want to import a job description directly from LinkedIn so that I do not need to manually copy and paste large amounts of text. As a user, I want to reuse previously saved job descriptions so that I can

	compare my resume against multiple opportunities efficiently. As a candidate, I want to import my LinkedIn profile information so that I can quickly generate or update my resume without rewriting existing experience.
Tailored Document Generation	As a candidate, I want to generate a tailored resume for a specific position so that I can improve my chances of passing Applicant Tracking Systems. As a user, I want the platform to generate a matching cover letter based on my resume and the target role so that I can prepare a complete application package quickly.
User Feedback & Evaluation Channel	As a user, I want to provide feedback after completing an analysis so that I can help improve the platform. As a developer, I want structured user feedback to evaluate future improvements and identify recurring usability issues.

0.7.1 [Feature 1]

PDF-to-Text Parser (Completed)

Rather than only supporting PDF resumes, the parser now accepts multiple commonly used document formats including PDF, Microsoft Word (DOCX), OpenDocument Text (ODT), Markdown, plain text, and LinkedIn profile exports. The ingestion pipeline automatically normalises these formats into a consistent internal representation before downstream analysis, allowing users to work with resumes originating from a variety of authoring tools without requiring manual conversion.

A robust PDF ingestion pipeline built on PyMuPDF that extracts raw resume text and preserves section hierarchy (Education, Experience, Projects, Skills). EasyOCR is used as the fallback for other formats to be parsed. The parser handles multi-column layouts, bullet point structures, and varied formatting styles to ensure clean, structured text is passed downstream to the RAG pipeline.

Header
John W. Smith 2002 Front Range Way Fort Collins, CO 80525
Summary
Four years experience in early childhood development with a diverse background in the care of special needs children and adults.
Experience
• Determined work placement for 150 special needs adult clients. • Maintained client databases and records. • Coordinated client contact with local health care professionals on a monthly basis. • Managed 25 volunteer workers. • Coordinated service assignments for 20 part-time counselors and 100 client families. • Oversaw daily activity and outing planning for 100 clients. • Assisted families of special needs clients with researching financial assistance and healthcare. • Assisted teachers with managing daily classroom activities. • Oversaw daily and special student activities. 1999-2002 Counseling Supervisor, The Wesley Center, Little Rock, Arkansas. 1997-1999 Client Specialist, Rainbow Special Care Center, Little Rock, Arkansas 1996-1997 Teacher's Assistant, Cowell Elementary, Conway, Arkansas
Education
University of Arkansas at Little Rock, Little Rock, AR • BS in Early Childhood Development (1999)

0.7.2 [Feature 2]

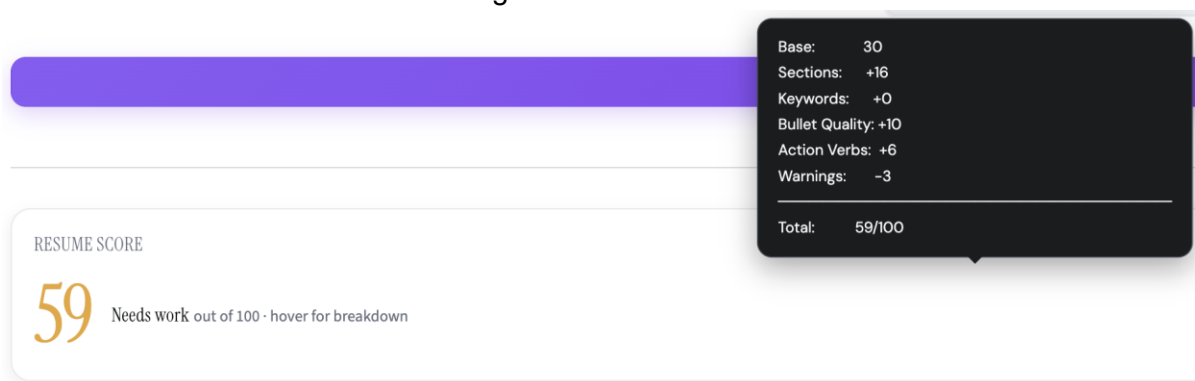
Vector Knowledge Base (Completed)

A RAG knowledge base built on ChromaDB, pre-loaded with authoritative resume framework documentation including the Google XYZ formula, the Rule of Three, ATS action verb lists, and STAR method guidelines. When a resume is uploaded, the system retrieves the most

semantically relevant frameworks and passes them as grounding context to the LLM, ensuring every suggestion is evidence-based rather than hallucinated.

0.7.3 [Feature 3] Resume Score & Impact Analyzer (Completed)

An LLM agent that critiques the uploaded resume and produces a composite score from 0 to 100, broken down across three dimensions: quantification of achievements (40 points), strength of action verbs and ATS alignment (30 points), and structural clarity (30 points). The analyzer identifies the weakest bullet points and explains precisely why they fall short, citing the relevant framework retrieved from the knowledge base.



0.7.4 [Feature 4] Side-by-Side Bullet Rewriter (Completed)

A web interface that presents each weak bullet point alongside an AI-generated rewrite using the Google XYZ formula. Users can accept the suggestion as-is, reject it, or manually edit the rewritten version inline. The goal is to make the improvement logic fully transparent, such that the user always sees the original, the suggestion, and the reasoning behind the change.

Signed in as jo Sign out

Review Suggestions

21 of 21 reviewed 19 accepted 2 dismissed

Accept all

Clear decisions

HIGHLIGHTED RESUME

2 / 2

Original stated delivery and personnel but lacked user count and time savings, applied Rule of 3 to add analyst scope and efficiency

Why: Original stated delivery and personnel but lacked user count and time savings, applied Rule of 3 to add analyst scope and efficiency

Engineered a modular scoring framework that evaluates securities across structure, sentiment, regime, and risk dimensions — automating feature engineering and candidate ranking to accelerate investment analysis.

WORK EXPERIENCE

Singapore Police Force — Secretary to Commander, ANK Division Apr 2022 — Apr 2024

First Sergeant — Personal Assistant to Commander & Deputy Commanders, Singapore's largest police division

Engineered an Excel-based automated manpower allocation system for large-scale policing operations, eliminating over 6 hours of manual scheduling effort per event and materially improving accuracy and auditability

Digitalized, structured, and governed divisional HR databases — enforcing data integrity, access controls, and compliance with confidentiality protocols across hundreds of personnel records

Produced executive briefs, analytical reports, and decision-support documentation for senior leadership; coordinated multi-stakeholder events and official engagements end-to-end

Self-Employed — Hardware Diagnostics — System Specification — Client Consulting Oct 2021 — Jan 2025

Designed, assembled, and optimized custom PC builds for gaming, productivity, and professional workloads; diagnosed and resolved complex hardware/software faults, maintaining a strong client retention record through reliable after-sales support

Managed end-to-end business operations — procurement, pricing, service delivery, and customer consulting — developing commercial acumen and independent technical problem-solving skills

VOLUNTEER & COMMUNITY

Disability Advocate — Facilitated a Singapore Handicap Basketball Association event at NUS; developing real-time subtitle tools and accessible mobile apps for youth with hearing loss

Education & Welfare: Teaching assistant for underprivileged kindergarten children; organized fundraising and donation drives for low-income families; provided care support at SPCK

AWARDS & RECOGNITION

SINDA SEA Excellence Award (2022) — Outstanding student with exemplary holistic achievements

MTCT Student Bursary Award — Awarded annually 2020-2023 and again 2025

American Mathematics Olympiad (2023) — Bronze Award

ORIGINAL VS. SUGGESTED REWRITE

ORIGINAL
SINDA SEA Excellence Award (2022) — Outstanding student with exemplary holistic achievements.

SUGGESTED REWRITE
Received SINDA SEA Excellence Award (2022) for outstanding holistic achievements among [N] candidates.

MENTOR'S SUGGESTION Dismissed

is this riyal

Dismissed — the suggestion above is the active recommendation again. Kept here as part of the review history.

Original stated award but omitted competitiveness

REWRITE DECISION

● NEEDS DECISION

Google XYZ

ORIGINAL	SUGGESTED REWRITE
Determined work placement for 150 special needs adult clients.	Coordinated strategic work placements for 150 adult clients with special needs, matching individual capabilities to employer requirements to achieve a [X%] successful placement rate.

💡 The original was a simple task description; I applied the Google XYZ framework by strengthening the action verb and adding a placeholder for the outcome/result.

0.7.5 [Feature 5]

ATS Keyword Matcher (Completed)

When the user provides a job description alongside their resume, this feature compares the resume's language against the job description using the RAG pipeline to surface missing ATS-critical keywords and technical skills. The system highlights terms that are present in the job description but absent from the resume, and suggests natural ways to incorporate them without keyword stuffing.

Joshua_Andrew_Resume_v2.docx.pdf

Change resume, job description, or provider

REVIEW

Suggestions

Keyword Gap

Extracted Sections

GENERATE

Tailored CV

Cover Letter

OPTIMIZE

Job Matching

INSIGHTS

Analytics

Mentor Feedback

83

Keyword Gap

COVERAGE

31%

11 of 35 JD keywords present in your resume

MISSING (24)

C# machine learning Generative AI Azure DevOps testing

troubleshooting data analysis application development

code walkthroughs security practices development processes

Data Science Software Engineering Artificial Intelligence

Information Technology AI tools AI services prompts

application support data retrieval reporting AI use cases

development practices documentation practices

PRESENT (11)

Python (4) Java (1) JavaScript (1) AI (5) SQL (2) Git (1)

GitHub (6) documentation (1) dashboards (2) automation (1)

Computer Science (2)

0.7.6 [Feature 6]

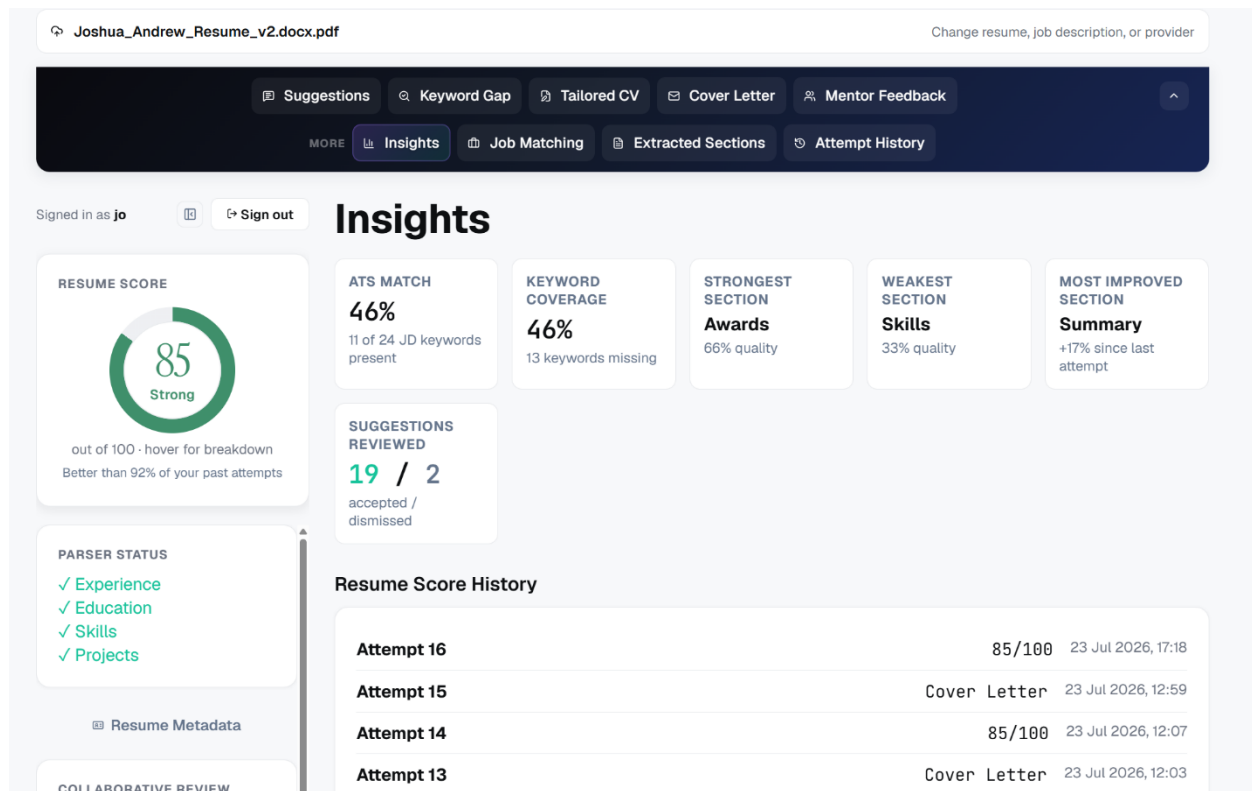
Agentic Self-Correction (Completed)

An iterative self-review pipeline where an independent Critic LLM evaluates the output produced by the primary Generator LLM before recommendations are presented to the user. The critic validates rewritten content against the original resume, identifying hallucinated metrics, unsupported claims, formatting inconsistencies, and violations of the project's factual-grounding principles. The system also includes a benchmarking framework that allows critic-enabled and critic-disabled configurations to be evaluated under identical conditions, enabling quantitative comparison of output quality, latency, and hallucination rates.

0.7.7 [Feature 7]

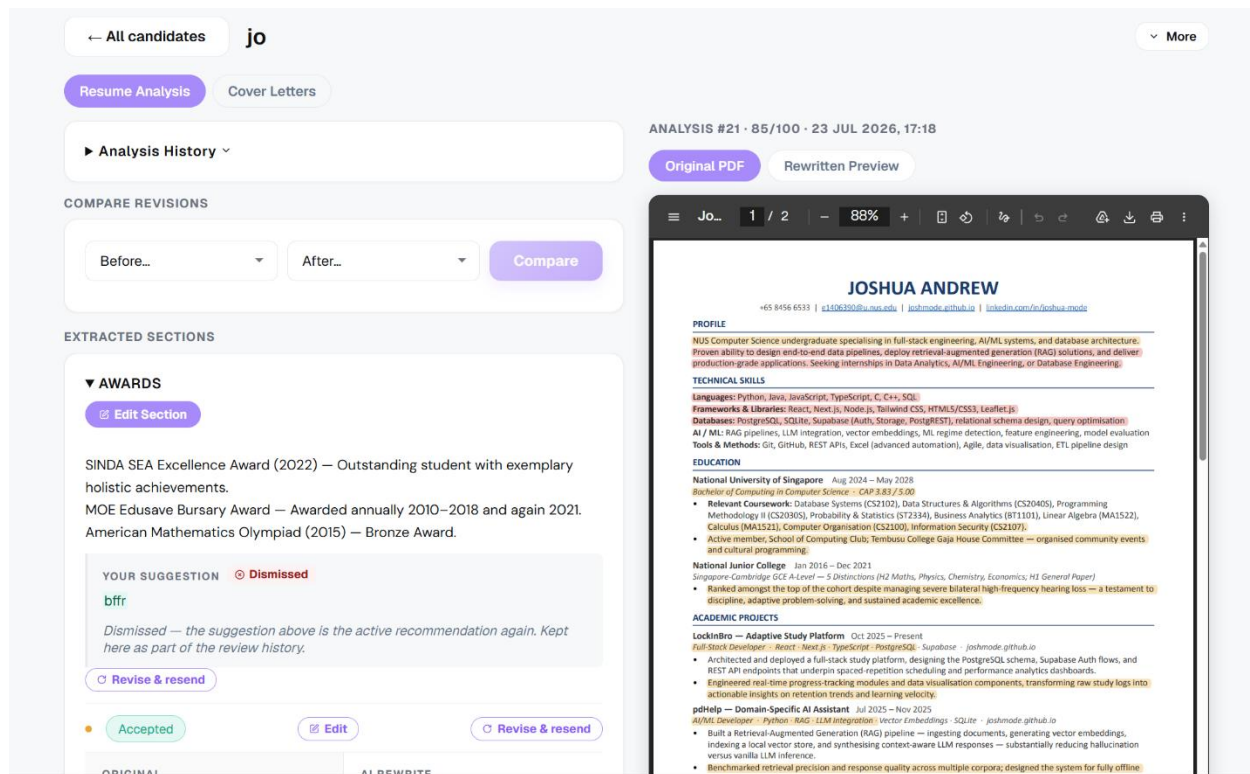
User Analytics Dashboard (Completed)

A persistent analytics workspace that tracks resume improvement across multiple submission iterations. Rather than displaying only the latest analysis, the dashboard maintains a complete history of previous analyses, resume scores, ATS performance, generated documents, mentor reviews, and candidate progress over time. Historical analyses may be revisited at any point, allowing users to observe long-term improvements, regressions, and recurring weaknesses throughout successive revisions.



0.7.8 [Feature 8] Mentor Review Workspace (Completed)

A collaborative mentor workspace designed to support structured resume coaching. Mentors may review candidate submissions, inspect previous analysis results, compare revisions using pull request-style difference visualisations, provide targeted feedback, and suggest improvements without directly overwriting the candidate's work. The workspace extends the project's human-in-the-loop philosophy beyond AI-generated recommendations by enabling professional review within the same workflow.



0.7.9 [Feature 9] Job Description & LinkedIn Integration (Completed)

The platform integrates directly with online job postings and LinkedIn exports to reduce manual data entry when tailoring applications. Users may import job descriptions directly from supported sources, analyse their resumes against target roles, and import profile information to accelerate resume construction while maintaining user oversight throughout the process.

0.7.10 [Feature 10] Tailored Document Generation (Completed)

Following analysis, the platform generates role-specific resumes and cover letters that incorporate accepted improvements and job-specific context while preserving the candidate's original experience. Generated documents may be exported in both Microsoft Word and PDF formats for immediate submission.

0.7.11 [Feature 11] User Feedback & Evaluation Channel (Completed)

An integrated feedback mechanism allows users to voluntarily submit qualitative feedback regarding analysis quality, usability, and overall experience. Feedback is collected only with explicit user consent and is intended to support ongoing system evaluation and future improvements.

INSIGHTS

Analytics

Mentor Feedback

✓	2026-07-21	73% GOOD	61% FAIR	75% GOOD	100% EXCELLENT	—	55% FAIR	33% NEEDS WORK
✓	#9 2026-07-21	73% GOOD	61% FAIR	75% GOOD	100% EXCELLENT	—	55% FAIR	33% NEEDS WORK
✓	#10 2026-07-22	66% GOOD	66% GOOD	55% FAIR	100% EXCELLENT	33% NEEDS WORK	33% NEEDS WORK	33% NEEDS WORK

► Technical Details

OPTIONAL EVALUATION

Share anonymised confidence feedback without including your resume content.
☐ I consent to store this evaluation response.

Confidence in these recommendations

Select a rating

Optional qualitative feedback

Submit Feedback

0.8 Initial Planning, Timeline & Development Blueprint

Milestone 1

Goal: Establish the foundational pipeline and prove the full stack works end-to-end.

Task	Description	Owner
Project Setup	GitHub repo, folder structure, virtual environment	Joshua Sijia
PDF-to-text parser	Extract and clean resume text from uploaded PDFs using PyMuPDF	Sijia
Vector knowledge base	Load XYZ formula, Rule of Three, and ATS action verb docs into ChromaDB	Joshua
LLM integration	Connect OpenAI API, pass resume text + retrieved context, return structured feedback	Joshua
Resume Score	0–100 score broken down by quantification, action verbs, and structure	Joshua
Streamlit frontend	PDF upload, extracted text display, raw AI feedback output	Sijia
Documentation	README, system architecture diagram, and demo video	Joshua Sijia

Milestone 2

Goal: Full RAG pipeline operational with core features and an improved user interface.

Task	Description	Owner
Full RAG pipeline	Retrieval + augmentation fully integrated; all suggestions grounded in retrieved context	Joshua
Impact Analyzer	LLM agent critiques bullet points and flags missing quantification and weak action	Joshua

	verbs	
Side-by-side bullet rewriter	Before/after display with accept, reject, and manual edit controls per suggestion	Sijia
Contact information detector	Auto-extract name, email, phone, and LinkedIn URL from resume text	Sijia
Section parser	Identify and separate Education, Experience, Skills, and Projects sections	Sijia
System testing	End-to-end testing and bug fixes in preparation for Milestone 2 submission	Joshua Sijia

Milestone 3

Goal: ATS matching, self-correction loop, beta testing, and public deployment.

Task	Description	Owner
ATS keyword matcher	Compare resume against a provided job description and surface missing ATS-critical keywords	Sijia
Agentic self-correction	Critic LLM verifies Generator LLM output for hallucinations, formatting errors, and fabricated metrics	Joshua
User analytics dashboard	Provides resume score history, improvement trend chart, and section-level readability heatmap	Sijia
Beta testing with 5-10 peers	Think aloud sessions, collect feedback, iterate on UI/UX	Joshua Sijia
Deployment	Streamlit Cloud/Vercel (Frontend) + Railway (Backend)	Joshua
Comprehensive Documentation	README, Swagger API docs, architecture diagram updates	Joshua Sijia

Tech Stack

Frontend

React + Vite	The user interface is implemented as a React single-page application (SPA) built using Vite. React provides a responsive, component-based interface for resume analysis, mentor review, analytics visualisation, and document management, while Vite enables fast development builds and efficient production bundling. Unlike the earlier Streamlit prototype, the frontend is now fully decoupled from the analysis engine, allowing independent development, deployment, and scaling of the user interface.
React router	Client-side routing is used to provide dedicated workspaces for resume analysis, analytics, mentor review, authentication, account management, and generated documents without requiring full page reloads. This enables a significantly richer multi-page user experience than the original single-page prototype.
Recharts	Interactive charts visualise historical resume performance, analysis scores, mentor feedback, ATS improvements, and long-term candidate progression. Rather than displaying only a single resume score, the analytics workspace presents historical trends across multiple analyses, allowing users and mentors to monitor improvement over time.

Backend/Parsing

PyMuPDF (fitz)	extracts text, bounding boxes, and page geometry directly from the uploaded PDF. Used both for the initial text extraction (parser.py) and for drawing highlight annotations back onto the PDF for the linked review view.
easyocr + pdf2image	fallback OCR pipeline for resumes that are scanned images rather than text-based PDFs. Only invoked when PyMuPDF detects fewer than 10 usable lines of extracted text.
Regex-based section & contact detection	A hand-tuned set of patterns and section-alias mappings that normalises the many real-world ways resumes label their sections into a consistent internal schema.
SQLite	SQLite provides persistent storage for user accounts, encrypted API keys, saved analyses, mentor feedback, job

	descriptions, generated documents, and application metadata. The lightweight embedded database simplifies deployment while providing sufficient functionality for the project's intended scale.
Express.js	Express serves as the public-facing REST API responsible for user authentication, request validation, job orchestration, document management, mentor workflows, and communication with the analysis engine. It also implements rate limiting, caching, retry logic, and secure API key management, separating application logic from the AI analysis pipeline.

RAG/AI Pipeline

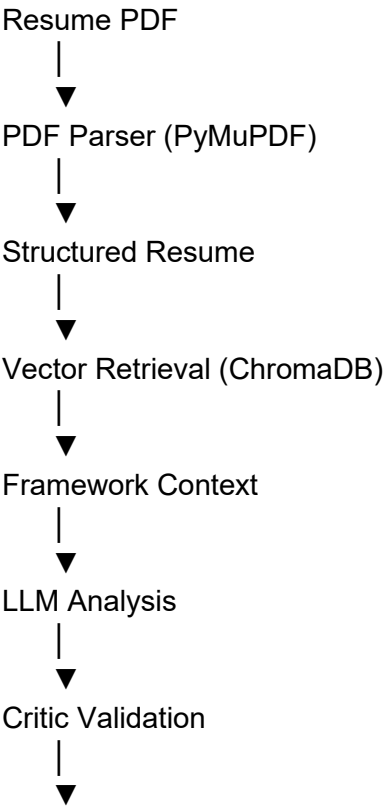
Flask	The AI analysis pipeline is implemented as a dedicated Flask service that operates independently of the public API. The engine is responsible for document parsing, resume analysis, Retrieval-Augmented Generation, LLM orchestration, document generation, and export functionality. Isolating the analysis engine behind an internal API improves modularity, simplifies deployment, and prevents computationally intensive workloads from directly affecting the public web application.
ChromaDB	Local vector database storing the "gold standard" resume frameworks (Google XYZ formula, Rule of Three, STAR method, ATS action verb lists). Queried per bullet before any rewrite is generated, so every suggestion is grounded in retrieved guidance rather than model intuition alone.
sentence-transformers	Generates the embeddings used to retrieve the most relevant framework documents from ChromaDB for a given bullet.
Multi-provider LLM router, Bring-Your-Own-Key	A provider-agnostic interface supporting Google Gemini, Anthropic Claude, OpenAI ChatGPT, and local models via Ollama, selectable per session from the sidebar. This keeps the project usable even if any single provider's API key, quota, or availability changes. Openrouter was chosen as default provider at website launch for its generous free-tier API. Rather than relying solely on globally shared API credentials, each authenticated user may securely configure their own provider API keys. Keys are encrypted using AES-

	256-GCM before storage and decrypted only when individual requests are executed, ensuring complete isolation between users while allowing independent provider selection.
Agentic self-correction	An optional second LLM pass that checks each rewrite for fabricated metrics or hallucinated numbers before it reaches the user, falling back to the original bullet if a rewrite cannot be verified safe.

Document Generation

python-docx	converts the generated Markdown CV/cover letter into a formatted, two-page-targeted Word document.
markdown-pdf	converts the same Markdown output into a downloadable PDF.

0.9 Architecture Overview



Interactive Review



CV / Cover Letter Generator

1. Introduction

The rapid adoption of Applicant Tracking Systems (ATS) and AI-assisted recruitment has fundamentally changed how resumes are evaluated. Before a recruiter ever reads a candidate's application, it is increasingly likely to be filtered, ranked, or summarized by automated software. Consequently, applicants have begun relying on Large Language Models (LLMs) to optimize their resumes for these systems.

While recent advances in generative AI have made resume assistance significantly more accessible, they have also introduced a new set of problems. Many existing AI-powered resume tools function as opaque black boxes: they generate polished-looking rewrites without explaining why those changes were made, often relying entirely on the internal reasoning of the language model. More critically, they frequently hallucinate—fabricating performance metrics, inventing technical achievements, or introducing irrelevant industry terminology that was never present in the original resume.

For a document intended to represent a person's professional experience, such hallucinations are more than a technical inconvenience—they undermine credibility and can expose applicants to uncomfortable questioning during interviews.

RAGsToRiches was developed to address these shortcomings through a fundamentally different architectural approach.

Instead of relying solely on prompt engineering, the system first retrieves relevant resume-writing frameworks from a curated semantic knowledge base before any generation occurs. These retrieved frameworks—including but not limited to the Google XYZ Formula, the STAR Method, the Rule of Three, and curated ATS-oriented writing principles—are injected into the prompt as contextual evidence (via a RAG pipeline), ensuring that every rewrite is grounded in recognized resume-writing standards rather than unconstrained language generation.

The result is a resume analysis platform that prioritises transparency, traceability, and factual integrity over superficial fluency. Rather than asking users to blindly trust an AI-generated suggestion, the system explains what changed, why it changed, and which writing framework informed the recommendation.

2. Motivation

Professional resume coaching has traditionally been expensive, placing high-quality personalised feedback beyond the reach of many students and early-career professionals. Existing commercial platforms frequently gate meaningful features such as resume scoring, ATS optimisation, or detailed rewrite suggestions behind recurring subscription models.

Although free AI tools exist, they often suffer from several recurring limitations:

- Generic recommendations that are not tailored to the uploaded resume.
- Lack of transparency regarding how suggestions were generated.
- Hallucinated achievements and fabricated quantitative metrics.
- Inconsistent behaviour across different language model providers.
- Limited support for offline or privacy-sensitive workflows.

These limitations become particularly significant when resumes contain confidential information, unpublished projects, or proprietary work experience that candidates may not wish to upload to external cloud services.

RAGsToRiches was designed around a different philosophy: that high-quality resume coaching should be transparent, explainable, and accessible without requiring expensive subscriptions or sacrificing data privacy.

Rather than attempting to replace professional career advisors, the system serves as an intelligent assistant that helps users understand established resume-writing principles and apply them consistently to their own experience. By grounding every recommendation in established knowledge instead of relying exclusively on model intuition, the platform encourages learning rather than passive acceptance of AI-generated content.

3. Design Philosophy

RAGsToRiches was designed around five core principles that guided every major architectural and engineering decision throughout development. Rather than optimizing solely for fluent AI-generated output, the system prioritizes transparency, factual integrity, accessibility, privacy, and long-term maintainability. These principles influenced every stage of the pipeline: from document parsing and semantic retrieval to prompt construction, session management, and deployment.

Transparency over Black Boxes

Modern AI systems often generate convincing outputs without providing any explanation of how those conclusions were reached. While this may be acceptable for conversational assistants, it is problematic for professional documents where every statement represents a candidate's own experience and achievements.

RAGsToRiches therefore prioritises transparency throughout the analysis pipeline. Every significant recommendation is accompanied by the reasoning behind it, including the resume-writing framework that informed the suggestion. Original and rewritten bullet points are presented side-by-side, allowing users to clearly understand what changed and why, rather than treating the language model as an infallible authority.

The objective is not merely to generate stronger resumes, but to help users understand the principles behind effective professional writing.

Evidence-Based & Factually Grounded Generation

Large Language Models possess extensive internal knowledge, but relying solely on model intuition often results in inconsistent recommendations or, more critically, fabricated information.

Rather than generating advice from the language model alone, RAGsToRiches employs Retrieval-Augmented Generation (RAG). Before any rewrite is produced, the system retrieves semantically relevant resume-writing frameworks, including the Google XYZ Formula, the STAR Method, the Rule of Three, and ATS-oriented writing principles, from a curated vector knowledge base. These retrieved frameworks are then incorporated into prompt construction,

grounding every recommendation in recognised resume-writing standards instead of unconstrained generation.

Equally important is the system's commitment to factual integrity. RAGsToRiches deliberately avoids inventing achievements, statistics, or performance metrics. When a resume bullet lacks measurable outcomes, the system inserts explicit placeholders (for example, **[X%]** or **[N users]**) and encourages users to replace them with genuine values obtained from their own work. This preserves the authenticity of the resume while still demonstrating where stronger evidence could be incorporated.

Privacy-First by Design

Resumes frequently contain sensitive personal information, including employment history, educational records, contact details, and proprietary project experience. Recognising this, the system was designed to support privacy-conscious workflows from the outset.

Through its provider-agnostic routing layer, users retain complete control over where their data is processed. They may choose between cloud-hosted providers such as Google Gemini, Anthropic Claude, and OpenAI, or execute the entire analysis pipeline locally using Ollama-hosted models. This flexibility enables sensitive resumes to remain entirely on the user's own machine whenever privacy is a priority.

Rather than locking users into a single proprietary AI provider, the architecture allows them to balance performance, privacy, availability, and cost according to their individual requirements.

Accessible AI for Everyone

High-quality resume coaching has traditionally been expensive, with many commercial platforms placing detailed feedback, ATS optimisation, and rewrite suggestions behind recurring subscription fees.

RAGsToRiches was developed with the belief that evidence-based career guidance should be accessible to everyone, particularly students, graduates, and early-career professionals. As an open-source project, the platform is designed to be freely available without restricting its core functionality behind artificial paywalls. Support for both cloud-hosted and locally executed language models further ensures that users can choose an approach that best matches their financial, technical, and privacy requirements.

Accessibility is therefore treated not simply as a pricing decision, but as a fundamental design principle.

Modular & Extensible Architecture

The project adopts a highly decoupled architecture that separates document ingestion, semantic retrieval, analysis, routing, and presentation into independent modules with clearly defined responsibilities.

This separation of concerns improves maintainability, simplifies debugging and testing, and allows individual subsystems to evolve independently. Improvements to the parser do not require modifications to the retrieval pipeline; additional language model providers can be introduced without changing the analysis engine; and future document formats or frontend implementations can be integrated without fundamentally redesigning the system.

This modular architecture also underpins the project's long-term vision. Planned extensions—including expanded retrieval corpora, fine-tuned resume models, collaborative review workflows, and additional deployment targets—can all be incorporated with minimal disruption to the existing codebase.

Human-AI Collaboration over Agentic Automation

Although RAGsToRiches leverages Large Language Models to analyse resumes and generate evidence-based recommendations, the system was never intended to replace human judgement. Resume writing is inherently personal and context-dependent, and no language model can fully understand an individual's career aspirations, professional experiences, or the nuanced expectations of every recruiter and hiring manager.

Consequently, the platform adopts a collaborative review philosophy that combines AI-assisted analysis with human expertise. Candidates retain complete control over every AI-generated recommendation through an interactive approval workflow, while mentors are able to participate directly in the refinement process through a dedicated collaborative review workspace. Rather than overwriting candidate content, mentors can review historical analyses, compare revisions through pull request-style difference visualisations, highlight specific changes, and provide structured feedback alongside the system's own recommendations.

This collaborative workflow positions the language model as an intelligent assistant rather than an autonomous decision-maker. AI accelerates the identification of weaknesses, retrieval of relevant writing frameworks, and generation of candidate improvements, while candidates and mentors remain responsible for evaluating those recommendations within the broader context of career goals, industry expectations, and personal experience.

By deliberately combining automated analysis with professional human review, RAGsToRiches seeks to produce recommendations that are not only technically well-founded, but also contextually appropriate, trustworthy, and ultimately aligned with the candidate's own intentions.

Collectively, these six principles define the identity of RAGsToRiches. Rather than functioning as another opaque AI writing assistant, the system combines deterministic software engineering with Retrieval-Augmented Generation to produce recommendations that are transparent, evidence-based, privacy-conscious, freely accessible, and architecturally maintainable.

4. Project Scope

RAGsToRiches is designed as an end-to-end resume intelligence platform that guides users from an uploaded resume to evidence-based application documents through a transparent, AI-assisted workflow. Rather than functioning as a general-purpose chatbot, the system is intentionally scoped around a single problem domain: helping candidates improve the quality, impact, and role-specific relevance of their professional application materials using established resume-writing frameworks.

The application accepts resumes in multiple commonly used formats—including PDF, Microsoft Word (DOCX), OpenDocument Text (ODT), Markdown, plain text, and LinkedIn profile exports—and optionally one or more target job descriptions. From these inputs, the system extracts structured resume content, performs semantic analysis, retrieves relevant writing frameworks from a vector knowledge base, evaluates Applicant Tracking System (ATS) compatibility, generates targeted rewrite suggestions, and produces tailored resumes and cover letters aligned to specific job applications.

Unlike many commercial resume tools that automatically rewrite documents in their entirety, RAGsToRiches deliberately keeps users involved throughout the revision process. Every recommendation is presented individually, allowing users to review, accept, reject, or manually modify suggestions before they are incorporated into generated documents. This human-in-the-loop workflow is further extended through collaborative mentor review, enabling professional feedback to complement AI-generated recommendations while ensuring that ownership of the final document always remains with the candidate.

The current implementation focuses on delivering a comprehensive resume analysis and application optimisation platform while remaining deliberately scoped around the recruitment preparation process rather than the broader hiring ecosystem. The platform assists users in producing stronger application materials but does not attempt to replace recruitment platforms, applicant tracking systems, or hiring workflows.

In Scope

The current implementation includes:

- Secure multi-user account management with JWT-based authentication and persistent user workspaces.
- Uploading and parsing resumes in PDF, DOCX, ODT, Markdown, plain text, and LinkedIn export formats.

- Robust preprocessing, OCR fallback, and section normalisation to improve parser reliability across varying resume layouts and document types.
- Retrieval-Augmented Generation (RAG) using a persistent ChromaDB vector database containing established resume-writing frameworks.
- Resume quality evaluation through multiple analysis stages, including structural assessment, ATS keyword matching, framework-grounded rewrite generation, and agentic self-correction.
- Side-by-side comparison of original and rewritten bullet points with interactive acceptance, rejection, and manual editing of individual suggestions.
- Resume scoring together with historical analysis records, progress tracking, and analytics across multiple submissions.
- Multi-provider language model support through a unified routing layer supporting both Bring Your Own Key (BYOK) and locally hosted language models.
- Secure storage of user-managed API keys using authenticated per-user encryption.
- Job description management, LinkedIn integration, and role-specific resume and cover letter generation.
- Mentor review workspaces supporting collaborative feedback, revision history, pull request-style difference visualisation, and structured candidate review.
- Export of generated resumes and cover letters in Microsoft Word and PDF formats.
- Consent-based user feedback collection to support future platform evaluation and improvement.

Deliberately Out of Scope

Several features remain intentionally excluded from the current implementation in order to maintain a clear focus on resume intelligence and application preparation.

These include:

- Automated job application submission.
- Functionality equivalent to a recruitment marketplace or public job board.
- Applicant management systems intended for enterprise recruiters.
- Real-time collaborative editing with simultaneous multi-user document editing.
- Automated interview scheduling or employer communication.
- Fine-tuned domain-specific language models trained exclusively for resume optimisation.
- Full integration with external recruitment ecosystems beyond the current LinkedIn import and job description workflow.

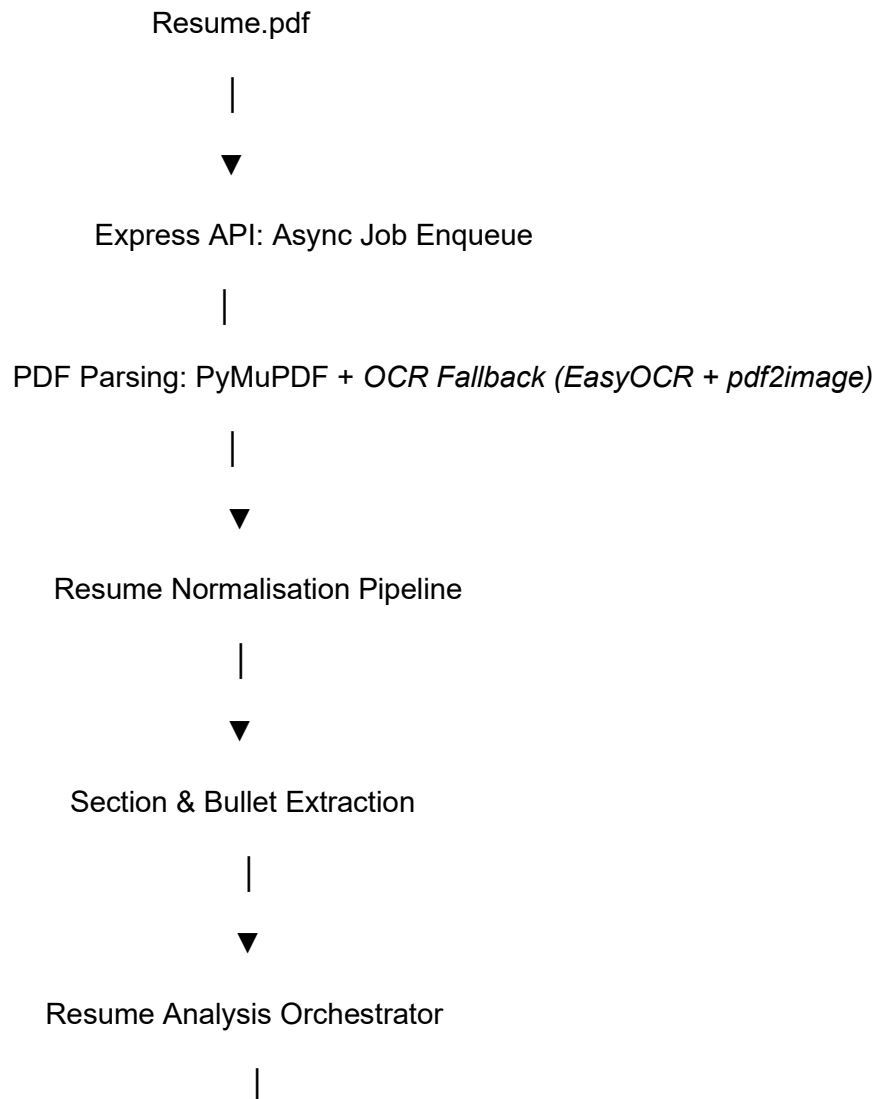
Restricting the project scope in this manner allows development effort to remain focused on improving analysis quality, retrieval effectiveness, explainability, collaborative review, and the overall reliability of the platform, rather than expanding into adjacent areas of the recruitment lifecycle.

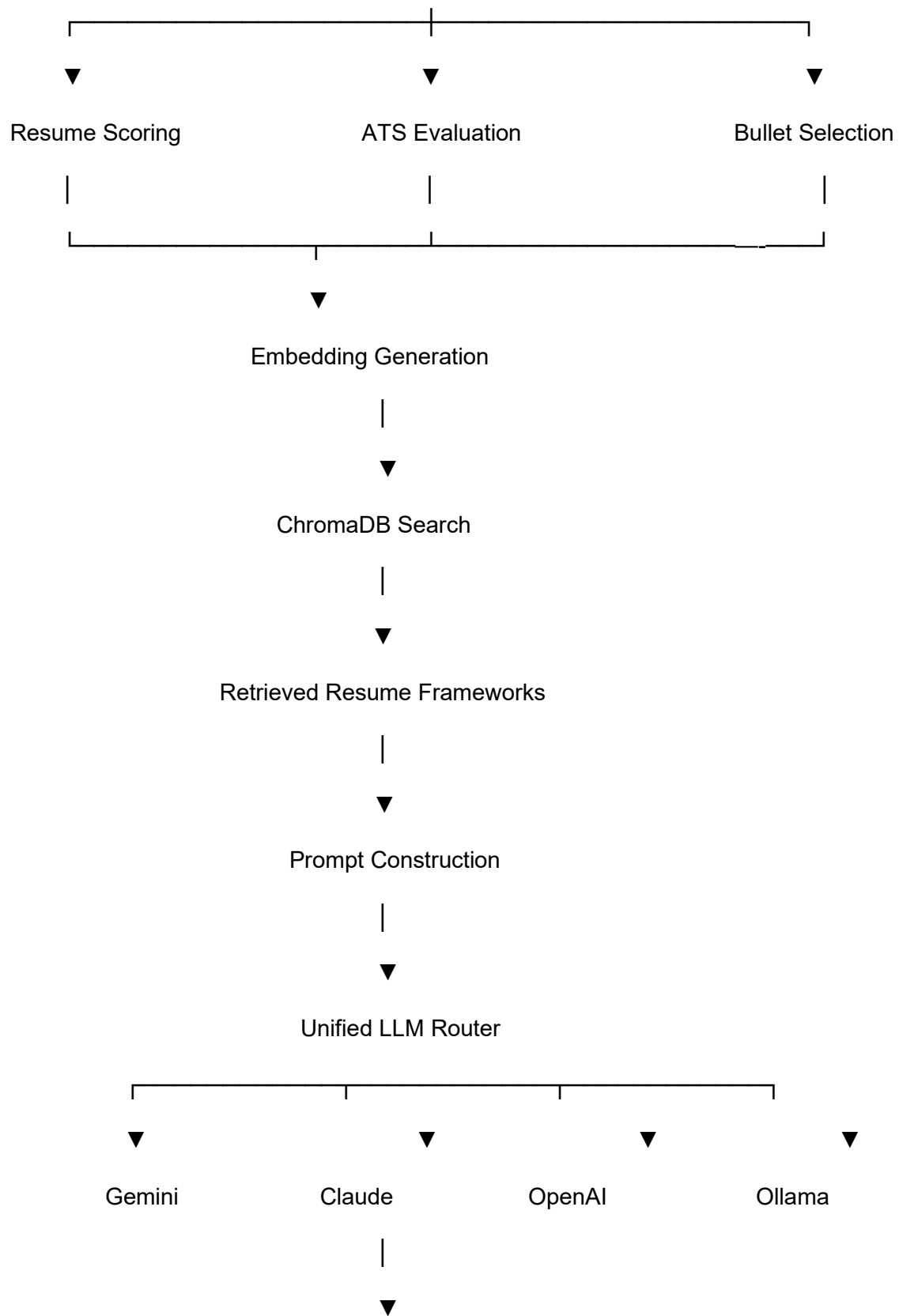
5. System Overview

At a high level, RAGsToRiches operates as a sequential processing pipeline that transforms an unstructured resume PDF into an interactive coaching dashboard.

Rather than treating the uploaded document as a single block of text, the application progressively enriches it through several independent stages. Each stage performs one specialised task before passing structured data to the next layer.

The complete workflow is illustrated below.





LLM Analysis, Critic Response Validation & Repair

|



React Layer: Interactive review, Express API: Persist to SQLite DB

|



Express API: Final CV/Cover Letter Generation

Although the pipeline appears linear, each subsystem is intentionally isolated from the others. The parser has no knowledge of the language model, the routing layer has no understanding of resume scoring, and the presentation layer does not perform any document analysis itself.

This strict separation of responsibilities makes the system easier to debug, easier to extend, and significantly more resilient when individual components fail.

For example:

- If an external LLM provider becomes unavailable, the parser still completes successfully and the user can continue reviewing extracted resume content.
- If semantic retrieval returns no sufficiently relevant framework, the analysis pipeline still produces baseline recommendations rather than terminating prematurely.
- If one language model encounters API rate limits, the routing layer will throw an exception, prompting the user to switch to another supported provider without affecting downstream modules.

This modular architecture was adopted early in development and refined throughout the project as additional features were introduced.

5.1 User Roles

Role	Description
Candidate	Job seekers who utilize the platform to upload their resumes for comprehensive AI analysis, iteratively refine bullet points through an interactive review process, generate tailored application materials, and collaborate with mentors.

Mentor	Career coaches or institutional staff members who access candidates' historical analysis records to provide structured, qualitative feedback and monitor professional development over time.
---------------	--

5.2 Core Workflows

Candidate Analysis Pipeline: A sequential workflow where users upload documents for parsing and semantic evaluation, review framework-grounded recommendations, and finalize modifications before exporting role-specific resumes or cover letters.

Rapid Cover Letter Generation: A streamlined utility allowing candidates to bypass full resume analysis to immediately produce a tailored cover letter by mapping resume content against a provided job description.

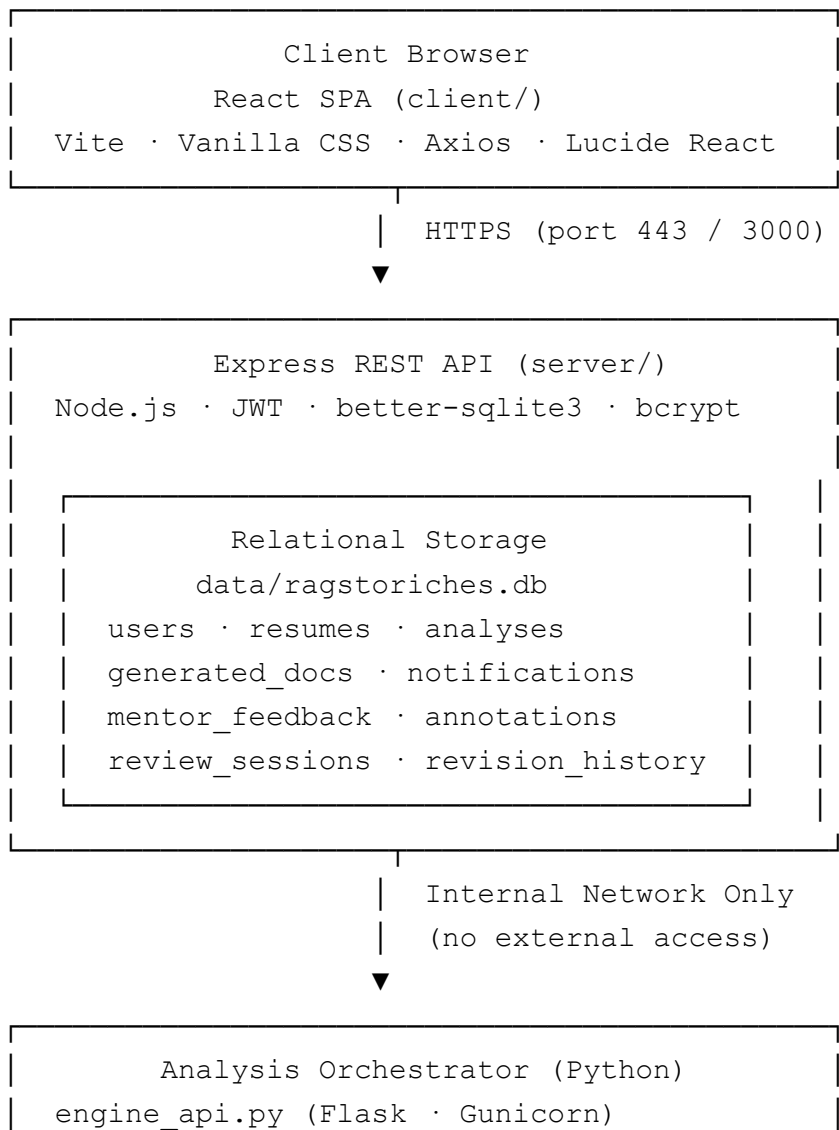
ATS Alignment Workflow: A comparative analysis that measures resume language against specific job requirements to surface keyword match scores, identify critical skill gaps, and highlight existing strengths.

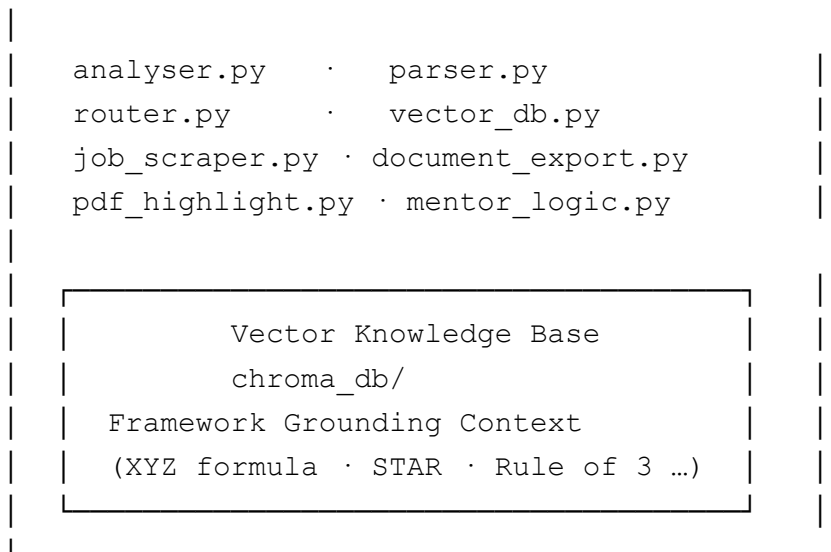
Collaborative Mentor Review: A structured feedback loop enabling mentors to inspect candidate submissions, propose targeted edits or section rewrites, and synchronize improvements directly with the candidate's workspace for immediate document persistence.

6. High-Level System Architecture

Eschewing a monolithic architecture, RAGsToRiches partitions core logic among three autonomous services that interact strictly via formalized boundaries. Such modularity represents a strategic engineering decision: the semantic analysis engine, the application backend, and the presentation tier evolve through independent lifecycles with distinct operational demands. Decoupling these layers ensures that any single component may be optimized, scaled, or substituted without compromising overall system integrity.

6.1 Architecture Overview





Each component maintains a strict separation of concerns. The Express API serves as the exclusive gateway for client requests, while the Python-based analysis engine remains isolated from the public network for enhanced security.

6.2 Presentation Layer — React SPA (client/)

Built on React 18 and Vite, the presentation tier is delivered as static assets by the Express server to provide a high-fidelity user experience.

Key responsibilities include:

- Rendering interactive dashboards for analysis review, tailored generation, historical tracking, and collaborative mentor interactions.
- Orchestrating UI state through modern React hooks like `useState` and `useEffect`.
- Facilitating server communication via an Axios-driven REST client.
- Managing complex UI behaviors including dynamic card animations, sticky panels, and persistent notification indicators.
- Executing periodic background polling for asynchronous analysis status and unread system alerts.

The frontend acts strictly as an orchestration layer; it contains no business logic or sensitive database credentials, ensuring a clean decoupling from the backend services.

The selection of React enables the sophisticated functionality required for the mentor workspace, which necessitates complex state management for version comparisons and discussion threads that traditional server-side rendering cannot effectively support.

6.3 Application Layer — Express REST API (server/)

The Express REST API serves as the primary coordinator, bridging the client interface with internal Python services and secure persistence.

Core responsibilities include:

- Managing user authentication and lifecycle via JWT and bcrypt-hashed security.
- Securing user-supplied LLM keys using authenticated AES-256-GCM encryption.
- Validating incoming requests prior to downstream processing.
- Operating a non-blocking job queue for intensive semantic analysis tasks.
- Implementing intelligent caching to optimize resource consumption for redundant analysis configurations.
- Enforcing granular rate limits to safeguard system availability across multiple traffic types.
- Persisting all relational metadata, user decisions, and feedback items within a localized SQLite store.
- Facilitating the collaborative lifecycle: enrolment, revision diffs, and structured feedback sessions.
- Surfacing real-time notification events to synchronize candidate and mentor workspaces.
- Proxying generation and highlighting requests to the isolated analysis engine.

Inference and heavy document parsing remain entirely delegated to the Python engine to maintain architectural purity.

6.4 Analysis Engine — Python Service (engine_api.py)

Implemented as a Flask application served via Gunicorn, the analysis engine provides specialized semantic services consumed by the Express backend.

Technical responsibilities include:

- Executing robust multi-format ingestion for PDF, Word, ODT, and LinkedIn profile data.
- Retrieving framework context through vector search against a curated ChromaDB store.
- Orchestrating model interactions via a provider-agnostic routing interface.
- Quantifying document quality across multiple evidence-based scoring dimensions.
- Validating rewrites through an agentic critic stage to ensure factual grounding.
- Performing semantic gap analysis for ATS alignment against specific job descriptions.
- Generating role-specific resumes and cover letters with Markdown, DOCX, and PDF export support.
- Processing PDF highlight overlays for bullet-linked review visualisations.

This service remains internal to the server environment, ensuring that analysis endpoints and sensitive parsing routines are shielded behind the authenticated application layer.

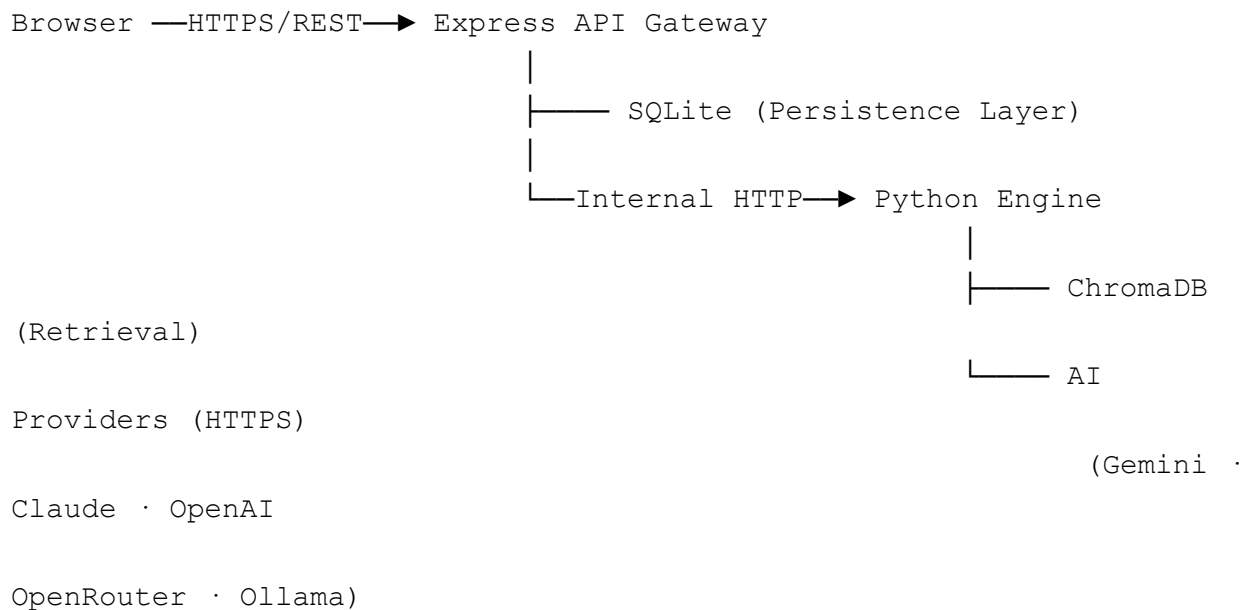
6.5 Persistence Layer

The application utilizes two distinct persistent storage solutions:

SQLite (`data/ragstoriches.db`) — Managed via `better-sqlite3` within the Express API. It stores all relational system state: accounts, resume history, decisions, and collaboration metadata. SQLite was selected for its zero-infrastructure profile and simplified backup capabilities, aligning perfectly with the synchronous API model.

ChromaDB (`chroma_db/`) — Managed by the Python analysis engine. This vector store persists embedded resume frameworks used for semantic grounding. It operates independently of the primary database and ensures context remains available across service restarts.

6.6 Communication Topologies



Strict communication boundaries ensure that the browser never interfaces with the analysis engine directly, and persistence is strictly isolated within the Express tier. This architectural discipline guarantees that each layer remains individually verifiable and deployable.

7. Technical Implementation

Although the user experiences RAGsToRiches as a single web application, internally it consists of several specialised processing pipelines that transform an uploaded resume into structured, explainable recommendations. Each pipeline was designed independently to maximise modularity while minimising coupling between components.

Unlike many AI-powered applications that simply upload a document and ask a language model to "improve it", RAGsToRiches performs significant deterministic preprocessing before any model is invoked. This reduces hallucinations, improves consistency, and ensures that the language model receives structured, meaningful context rather than an unprocessed document.

7.1 Document Ingestion Pipeline (parser.py)

The first stage of the application is responsible for converting an uploaded resume file into structured textual information suitable for semantic analysis.

Because resumes vary significantly in formatting, the ingestion layer cannot assume a consistent document structure. Modern resumes frequently contain multi-column layouts, tables, icons, floating text boxes, custom typography, embedded images, letter-spaced section headings, and decorative separators. Traditional PDF parsers often fail when confronted with these layouts, producing fragmented sentences, incorrectly ordered text, or entirely missing sections.

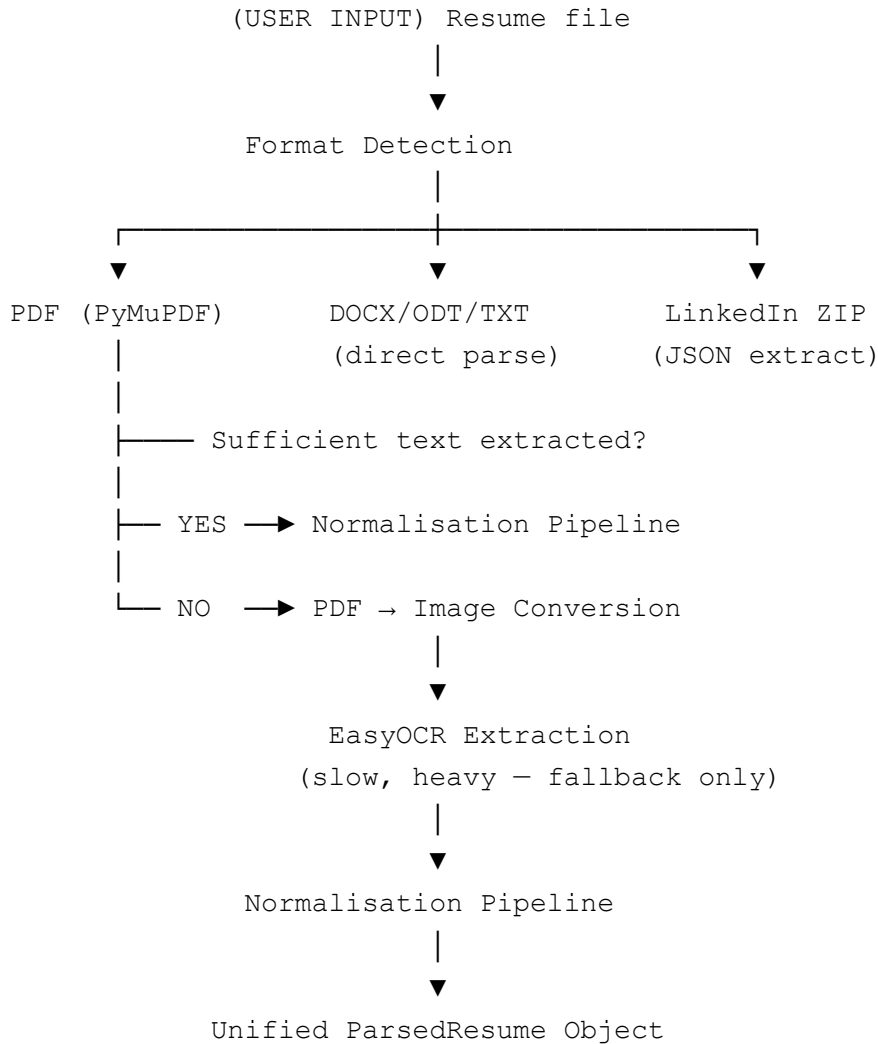
Rather than relying on a single extraction strategy, RAGsToRiches employs a layered parsing pipeline that progressively attempts more robust extraction methods when simpler approaches fail.

The parser handles six input formats:

Format	Method
PDF (text-based)	PyMuPDF (fitz) — primary extraction path
PDF (scanned / image-only)	EasyOCR + pdf2image — lazy-loaded OCR fallback
DOCX	python-docx
ODT	odfpy
Plain text / Markdown	Direct line-by-line parsing

LinkedIn export ZIP	Structured JSON extraction
---------------------	----------------------------

The overall parsing workflow is illustrated below.



PyMuPDF is used as the primary extraction engine due to its excellent performance and ability to preserve much of the original document structure. If extraction quality is insufficient — for example, when processing scanned resumes or image-based PDFs — the parser falls back to an OCR pipeline built on EasyOCR and pdf2image.

The OCR reader is lazy-loaded: it is only initialised when a scanned PDF genuinely requires it. Because `easyocr` transitively imports `torch`, which alone requires several hundred megabytes of memory, importing it unconditionally at startup would significantly increase baseline memory consumption for every engine process — even when no scanned PDFs are ever submitted. Normal (non-scanned) resume uploads never touch `torch`.

python

```
_reader = None
def _get_reader():
    """Lazy-loads and caches the EasyOCR reader instance."""
    global _reader
```

```

    if _reader is None:
        _reader = easyocr.Reader(["en"])
    return _reader
def _is_scan(lines):
    """Determines if the extracted text is too sparse to be a text-based
    PDF."""
    useful = [l for l in lines if len(l.strip()) > 3]
    return len(useful) < 1

```

The parsing output is a structured `ParsedResume` object that all downstream components consume:

python

```

@dataclass
class ParsedResume:
    raw_text: str
    contact: dict
    sections: dict[str, list[str]]    # canonical section name → bullet list
    warnings: list[str]

```

7.2 Resume Normalisation

Extracting text from a document is only the first step. Raw output is frequently inconsistent, containing fragmented whitespace, duplicated lines, irregular bullet formatting, or non-standard section titles. Before any semantic analysis occurs, the extracted content undergoes a series of deterministic preprocessing steps:

- Removal of duplicated text fragments
- Whitespace normalisation
- Bullet point reconstruction
- Header standardisation
- Contact information extraction
- Section boundary detection
- Section title canonicalisation

For example, resumes rarely use identical section names. The following headings may all refer to professional experience:

- Professional Experience
- Employment History
- Relevant Experience
- Work Experience
- Career History
- Industry Experience

Internally, these are normalised into a common representation so that downstream analysis operates on a consistent document structure rather than arbitrary formatting choices.

python

```
SECTION_ALIASES = {
    "employment history": "EXPERIENCE",
    "career history": "EXPERIENCE",
    "professional experience": "EXPERIENCE",
    ...
}
```

Similarly, education, project, and skills sections are canonicalised before being passed to the analysis engine. Separating normalisation from semantic analysis greatly reduces prompt complexity and improves downstream consistency.

7.3 Bullet Segmentation

Most resume analysis occurs at the bullet level rather than the document level. One of the parser's primary responsibilities is therefore identifying individual resume bullets while preserving their relationship to their parent section.

Rather than treating an entire experience block as a single paragraph, the parser constructs a hierarchical representation:

Experience

```
|— Company
|— Role
|— Bullet 1
|— Bullet 2
└— Bullet 3
```

Projects

```
|— Project Name
|— Bullet 1
└— Bullet 2
```

This hierarchical representation enables later stages of the pipeline to evaluate, rewrite, and explain individual bullets independently while still retaining broader document context. It also allows the user interface to navigate suggestions one bullet at a time, rather than presenting overwhelming full-document rewrites.

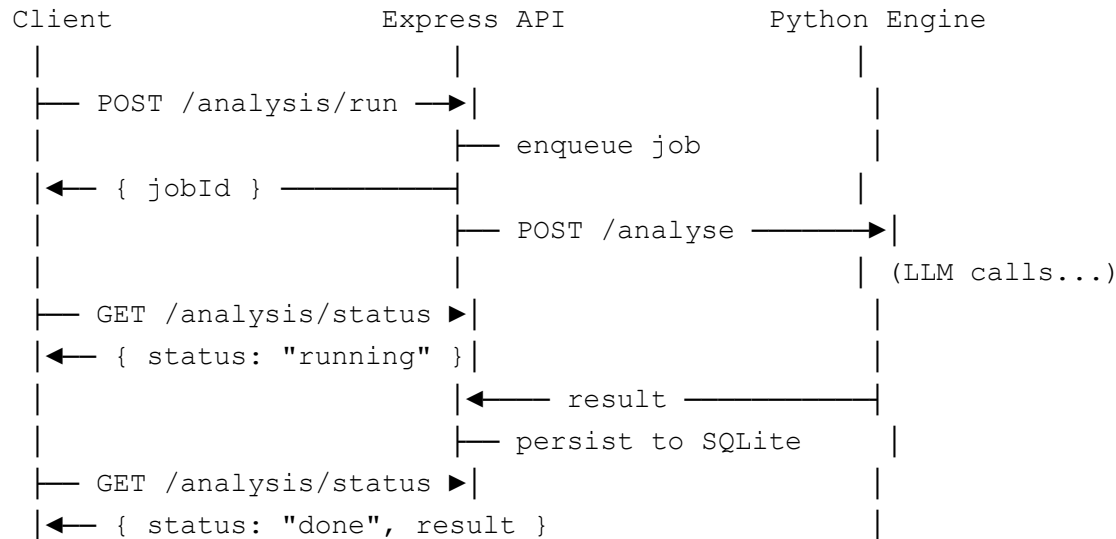
7.4 Express API Orchestration (server/)

Once a resume is submitted, the Express REST API is responsible for orchestrating the entire analysis lifecycle. Unlike a simple synchronous proxy, the API manages several cross-cutting concerns that would be impractical to handle at either the frontend or the engine level.

7.4.1 Asynchronous Job Queue

Resume analysis is a long-running operation: a 20-bullet resume with a full RAG pipeline can take 30–90 seconds depending on the chosen provider. Rather than holding an HTTP connection open for the duration, the Express API implements an asynchronous job queue.

The workflow is:



Each job is stored in the `analysis_jobs` table with its current status. The client polls `GET /api/analysis/jobs/:id` until the job reaches `done` or `error`. This design means that a page refresh during a running analysis does not lose the job — the client simply resumes polling.

7.4.2 Smart Result Caching

Running the same resume against the same job description with the same provider and critic settings multiple times wastes LLM API budget and adds unnecessary latency.

The API therefore checks for an existing row in the `analyses` table matching the same content hash, job description, provider, model, and critic configuration within 60 minutes. If a match is found, the cached result is returned immediately without invoking the engine.

7.4.3 Rate Limiting

Three independent rate limiter buckets prevent abuse without penalising ordinary usage:

Bucket	Scope	Limit
<code>generalLimiter</code>	All <code>/api/*</code> traffic	600 req / 15 min / IP
<code>pollLimiter</code>	Job status polling + PDF re-highlighting	90 req / min / IP

authLimiter / llmLimiter	Login attempts + LLM-spending endpoints	20 / 30 req / 15 min / IP
-----------------------------	--	---------------------------

Every 429 response includes a standard Retry-After header. The frontend's Axios client automatically retries on 429 with exponential backoff and jitter before surfacing an error, so a momentary limit is typically invisible to the user.

7.5 Authentication and API Key Management (server/routes/auth.js, server/userKeys.js)

RAGsToRiches supports two user roles — candidate and mentor — each with distinct permissions enforced at every relevant route.

Authentication is stateless JWT-based. Passwords are hashed with bcrypt before storage. Every protected route verifies the JWT and extracts the user's role before proceeding.

7.5.1 Bring-Your-Own-Key (BYOK)

Each user may save their own API key for Gemini, Claude, or OpenAI. These keys are encrypted at rest using AES-256-GCM. They are decrypted server-side only at the moment of an LLM call and are never returned to any client.

```
User saves key → AES-256-GCM encrypt → store in users table
LLM call      → decrypt in-memory   → forward to provider
                (key discarded after call)
```

Two users can each save a different Gemini key and their requests never cross. This replaced an earlier design where every saved key was written into one shared file, meaning whoever saved a key last silently became the key used by everyone.

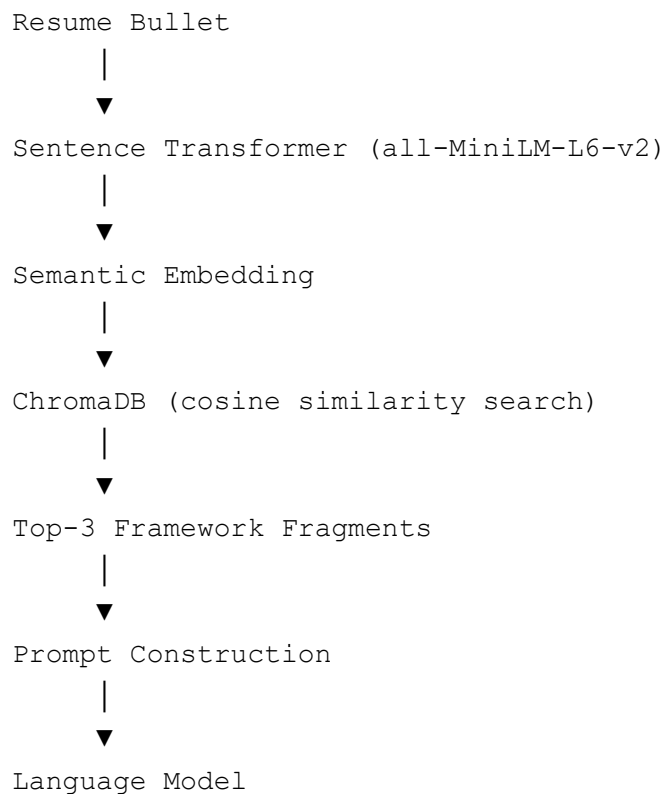
7.5.2 Pooled Default Free Tier

Users who choose "Default (Free)" share a single OpenRouter API key set by the operator. This uses OpenRouter's free-tier model pool, making the platform usable by visitors with no API key of their own.

7.6 Retrieval-Augmented Generation Pipeline (vector_db.py)

Traditional prompting approaches often embed lengthy resume-writing guides directly inside every request. While simple, this strategy scales poorly: long prompts increase token usage, reduce available context for resume content, and frequently introduce irrelevant guidance that has little bearing on the bullet currently under review.

RAGsToRiches instead employs Retrieval-Augmented Generation (RAG). Rather than storing resume-writing guidance inside prompts, the application stores it inside a persistent semantic knowledge base and retrieves only the most relevant fragments for each bullet.



Only the 3 most semantically relevant framework fragments are retrieved for each bullet. This substantially reduces prompt length while ensuring that every rewrite is grounded in guidance specifically applicable to the current piece of resume content.

python

```
frameworks = query_fw(bullet, n_results=3)
```

This substantially reduces prompt length while ensuring that every rewrite is grounded in guidance specifically applicable to the current piece of resume content.

Because retrieval occurs dynamically for every bullet, different sections of the same resume may receive different contextual guidance. A project description discussing technical implementation may retrieve the STAR framework, while a leadership experience may retrieve guidance on measurable organisational impact. This adaptive retrieval is considerably more flexible than static prompt templates.

7.6.1 Why Retrieval Instead of Prompt Engineering?

There are several engineering reasons for preferring RAG over prompt-embedded frameworks. First, prompt length grows linearly with the number of frameworks embedded directly into the prompt. As additional writing principles are introduced, prompt size quickly becomes inefficient. Second, most resume bullets require only a small subset of available guidance. Embedding every framework into every prompt wastes valuable context window capacity and often distracts the language model with irrelevant information.

Third, RAG forces the language model to ground every recommendation in retrieved industry-standard frameworks, ensuring these guidelines remain central to the generation process rather than being diluted or ignored during inference.

Finally, retrieval provides explainability. Because the retrieved framework is known before generation begins, the application can identify which writing principle influenced each recommendation rather than presenting opaque AI-generated advice.

7.6.2 Semantic Knowledge Base

The knowledge base forms the foundation of the application's explainability. Rather than storing example resumes or generated outputs, it contains curated resume-writing frameworks drawn from established best practices:

- Google XYZ Formula
- STAR Method
- Rule of Three
- ATS-oriented action verbs
- Quantification guidelines
- Impact-oriented writing principles

Each framework is embedded using a sentence-transformer model and stored in ChromaDB.

```
json
```

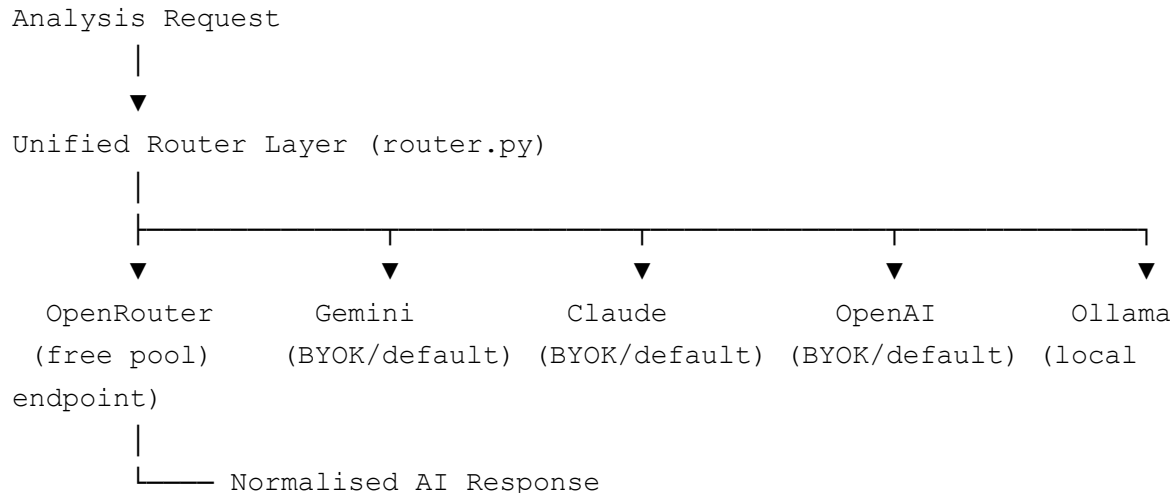
```
{
  "framework": "Google XYZ",
  "description": "Accomplished X, as measured by Y, by doing Z.",
  ...
}
```

7.7 Language Model Routing Layer (router.py)

One of the earliest architectural decisions made during development was to separate language model communication from the rest of the application.

Many LLM applications embed provider-specific SDK calls directly inside business logic. While sufficient for prototypes, this quickly becomes difficult to maintain as additional providers are introduced. Every new model requires changes throughout the codebase, resulting in duplicated logic for authentication, request formatting, error handling, and response parsing.

To avoid this, RAGsToRiches implements a dedicated routing layer responsible for all communication with external language models.



The routing layer performs several responsibilities simultaneously:

- provider selection based on user configuration
- authentication (API key resolution: BYOK, pooled operator key, or local)
- request construction in the format each provider requires
- retry logic: up to 5 attempts with exponential backoff and jitter on provider 429 / 5xx responses
- automatic fallback to openrouter/free if the pinned OpenRouter model has been deprecated
- rate-limit handling at the provider level
- response normalisation into a consistent internal format

Because every provider exposes different authentication methods, request formats, response schemas, and SDK conventions, abstracting these differences behind a common interface greatly simplifies the remainder of the application. The analysis engine interacts with a single interface regardless of which model ultimately generates the response.

7.7.1 Multi-Provider Support

Supporting multiple providers was not simply a convenience feature — it became an important engineering requirement throughout development.

Different language models exhibit different characteristics. Some produce more concise rewrites; others produce stronger reasoning. Some prioritise speed; others prioritise output quality. Local models provide significantly greater privacy but may require longer inference times depending on available hardware.

Rather than forcing users into a single ecosystem, the routing layer allows the application to support several providers simultaneously. Because all providers ultimately receive the same structured prompt produced by the analysis pipeline, switching providers requires no changes to parser logic, retrieval logic, or frontend behaviour.

This provider-agnostic design also makes the application more resilient to API changes, temporary service outages, and evolving pricing models — all of which were encountered during development.

7.7.2 Agentic Self-Correction (Critic)

An optional second LLM pass — "Agentic Self-Correction" — checks each rewrite for invented numbers, fabricated metrics, or hallucinated content before it reaches the user.

python

```
if use_critic:
    critic_prompt = (
        "You are a strict QA Critic for resume rewrites. "
        f"Original Bullet: {bullet}\n"
        f"Rewritten Bullet: {result.get('rewritten', bullet)}\n"
        "Did the rewrite invent any specific metrics, numbers, or percentages "
        "that were NOT in the original bullet? "
        "(Placeholders like [X%] are allowed). "
        "Return ONLY 'PASS' if it is safe, or 'FAIL: <reason>' if it "
        "hallucinated."
    )
```

The critic has its own independently configurable provider and model. It can be configured to share the main provider's credentials, or always run against a fixed cheap provider regardless of what the user chose for the main analysis — enabling every critic check to run on one fast, low-cost provider even when the main request uses BYOK Claude or ChatGPT.

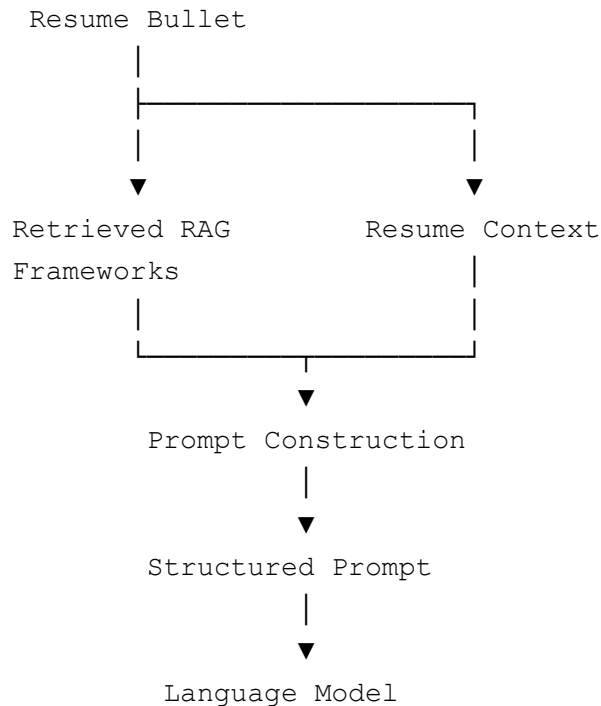
7.8 Prompt Construction

The quality of language model output depends heavily on the quality of the prompt supplied to it. Rather than constructing prompts from raw resume text alone, RAGsToRiches builds prompts by combining several independent sources of information.

Each prompt contains:

- the original resume bullet
- relevant surrounding context
- retrieved resume-writing frameworks
- ATS keywords to incorporate naturally (if a job description was supplied)
- explicit behavioural instructions
- formatting requirements
- validation constraints (including banned words and placeholder rules)

Conceptually, prompt construction follows a modular pipeline.



Separating prompt construction from retrieval allows each subsystem to evolve independently. Additional frameworks can be introduced without modifying parser logic, while prompt templates can be refined without rebuilding the vector database.

```
def rewrite_item(...) -> dict:
    """Rewrite a single bullet using RAG frameworks. Optionally run critic
    loop."""
    fw_ctx = "\n\n".join(f.document for f in frameworks)
    kw_hint = ", ".join(missing_kws[:12]) if missing_kws else "none"
    sys_prompt = (
        "You are an expert resume coach. Rewrite weak resume bullets into strong, "
        "ATS-optimised, results-driven statements using the STAR method "
        "(Situation, Task, Action, Result) or Google XYZ framework where "
        "applicable. "
```

```

"Write in a grounded, conversational tone – like a competent engineer
wrote it, "
"not a marketing copywriter. "
"CRITICAL – never invent, assume, or extrapolate any facts: no new
metrics, "
"percentages, team sizes, technologies, employers, or outcomes that are
not "
"explicitly stated in the original bullet. Use placeholders like [X%] or
[N users] "
"when a metric is clearly missing and note it in reasoning. "
"You MUST apply one of the provided writing frameworks. "
f"\n\nBANNED WORDS (never use these): {_BANNED_WORDS}\n"
"Always respond with valid JSON only, no markdown."
)

usr_prompt = (
f"FRAMEWORK GUIDANCE (apply the most relevant one):\n{fw_ctx}\n\n"
f"ATS KEYWORDS TO WEAVE IN NATURALLY (only if genuinely
relevant):\n{kw_hint}\n\n"
f"BULLET TO REWRITE:\n{bullet}\n\n"
"Respond with this exact JSON structure:\n"
'  "rewritten": "the improved bullet point",\n'
'  "reasoning": "one sentence: what was weak, what framework was applied,
what changed",\n'
'  "framework_used": "Google XYZ | STAR | Rule of 3 | Action Verb |
other",\n'
'  "severity": "red | yellow | green"\n'
"Severity guide – rate the ORIGINAL bullet, not the rewrite:\n"
"red = very weak passive language, missing action verb, or no discernible
impact.\n"
"yellow = structurally okay but missing a metric or could be stronger.\n"
"green = already strong; only minor polish applied."
)

```

7.9 Batched Bullet Rewriting

The original analysis pipeline sent one LLM request per resume bullet. A 20-bullet resume therefore meant 20 sequential round trips, each paying full network latency and provider queueing overhead.

The analysis engine now batches bullets into chunks and rewrites an entire chunk in a single LLM call. The same resume now requires significantly fewer calls with no reduction in per-bullet

quality: each bullet in a chunk still receives its own RAG framework context and is rewritten independently within the response.

If a chunk's response cannot be parsed back into exactly one result per bullet, that chunk automatically falls back to individual per-bullet calls. Batching therefore only ever adds speed — it never introduces a new failure mode.

7.10 Structured Response Parsing

Large Language Models are inherently probabilistic. Even when provided with identical prompts, they may produce outputs with slight formatting differences. Since downstream components require predictable data structures, the application requests responses in structured JSON formats wherever possible.

Where necessary, defensive parsing routines strip markdown fences, locate the JSON object within surrounding text, and validate the structure before it is accepted by the remainder of the pipeline.

```
def _parse_json(raw: str) -> Any:
    """Extract JSON from LLM output, stripping markdown fences and
    filler."""
    cleaned = raw.strip()
    if cleaned.startswith("```"):
        lines = cleaned.splitlines()
        if lines and lines[0].startswith("```"):
            lines = lines[1:]
        if lines and lines[-1].startswith("```"):
            lines = lines[:-1]
        cleaned = "\n".join(lines).strip()
    match = re.search(r'({.*})', cleaned, re.DOTALL)
    if match:
        cleaned = match.group(1)
    else:
        match = re.search(r'(\[.*\])', cleaned, re.DOTALL)
        if match:
            cleaned = match.group(1)
    return json.loads(cleaned)
```

Defective outputs are detected and handled gracefully rather than propagating errors throughout the application. This validation layer significantly improves robustness when interacting with probabilistic language models.

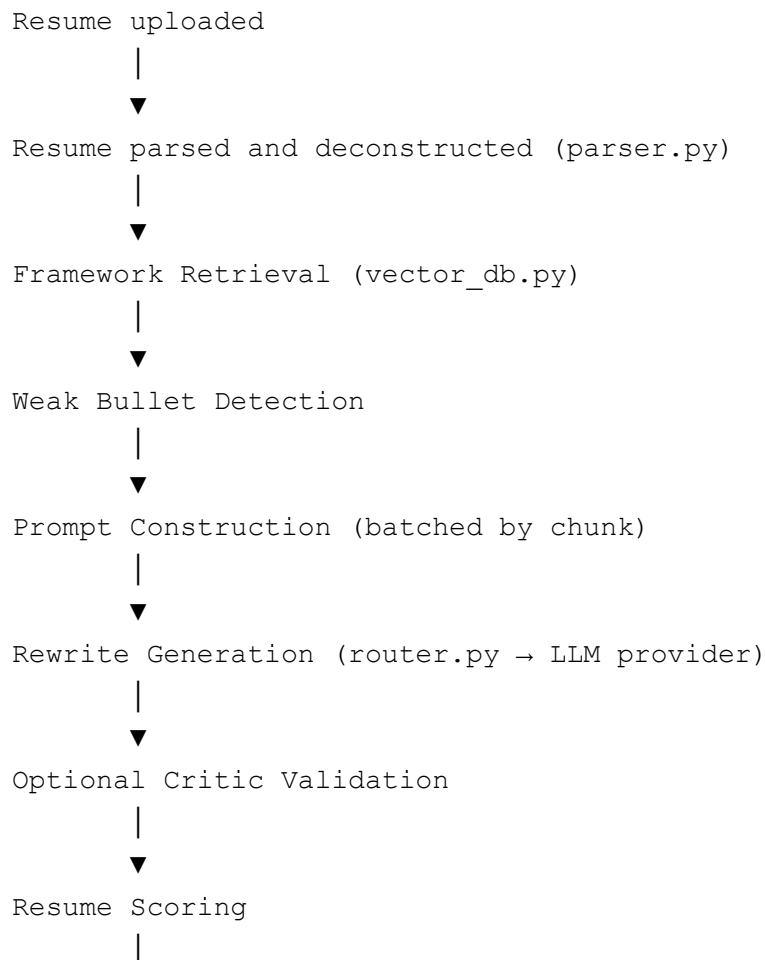
7.11 Resume Analysis Engine (analyser.py)

The analysis engine represents the central orchestration layer of the Python service. Unlike the parser, which performs deterministic preprocessing, the analyser combines deterministic algorithms with semantic reasoning performed by language models.

Its responsibilities include:

- coordinating resume scoring
- selecting bullets for improvement
- retrieving supporting frameworks
- constructing prompts
- batching and generating rewrite suggestions
- running the optional critic pass
- analysing ATS compatibility
- producing explanations and framework attributions
- coordinating output to the Express API

Rather than implementing these tasks independently, the analysis engine executes them as a coordinated workflow.



▼
ATS Keyword Gap Analysis (if JD supplied)



Structured result → Express API → SQLite

Keeping orchestration within a single module greatly simplifies control flow while allowing specialised components — retrieval, routing — to remain independent and interchangeable.

7.12 Resume Scoring

One of the application's primary objectives is to provide users with an interpretable assessment of resume quality rather than simply generating rewritten content. Instead of assigning an arbitrary score, the analyser evaluates several dimensions of resume writing commonly associated with effective technical resumes:

- measurable impact and quantification
- strength of action verbs
- structural clarity
- readability
- ATS keyword coverage
- evidence of quantifiable achievements
- score adjustments for accepted rewrites
- penalty points for parser warnings

```
def calc_score(resume, jd_kws, missing, rewrites) -> dict:
    bd = {"base": 30, "sections": 0, "keywords": 0,
          "bullet_quality": 0, "action_verbs": 0, "warnings": 0, "total":
0}
    # +8 per major section present (max 32)
    for sec in ("EXPERIENCE", "EDUCATION", "SKILLS", "PROJECTS"):
        if sec in resume.sections:
            bd["sections"] += 8
    # keyword coverage: up to +20
    if jd_kws:
        found = len(jd_kws) - len(missing)
        bd["keywords"] += int((found / len(jd_kws)) * 20)
    # bullet quality by severity: green=+2, yellow=+1, capped at 10
    # action verb usage: up to +8
    # warnings penalty: -3 per missing critical section
    ...
    bd["total"] = max(0, min(100, sum(v for k, v in bd.items() if k !=
"total")))
```

```
return bd
```

Scores are stored persistently in the analyses table and surfaced in the candidate's Insights dashboard across all historical attempts, enabling candidates to track improvement over time. Rather than functioning as an absolute measure of resume quality, the score serves as a diagnostic indicator that helps users identify which areas would benefit most from revision. Users are encouraged to interpret the score alongside the accompanying explanations rather than treating it as a standalone indicator of employability.

7.13 ATS Keyword Gap Analysis (job_scraper.py)

Modern recruitment workflows frequently involve Applicant Tracking Systems that perform automated filtering before resumes reach human recruiters. Simple keyword matching, however, is often insufficient: different organisations may describe similar technical skills using different terminology, while candidates may communicate equivalent experience using different wording.

To address this limitation, RAGsToRiches performs semantic ATS analysis rather than relying exclusively on literal substring matching. When a job description is supplied, the engine analyses both documents together to identify match percentages and skill gaps.

- relevant technical skills already represented in the resume
- important concepts absent from the resume
- the overall job match percentage
- keyword-level match coverage
- the target company name (auto-detected from the JD)
- opportunities for stronger alignment and tailoring suggestions

These results are stored for every analysis attempt and surfaced in Attempt History and the mentor's candidate view. Rather than encouraging indiscriminate keyword stuffing, the application recommends incorporating missing concepts only where they accurately reflect the candidate's genuine experience.

7.14 Document Generation (document_export.py)

Generated documents are assembled from several sources applied in order of precedence:

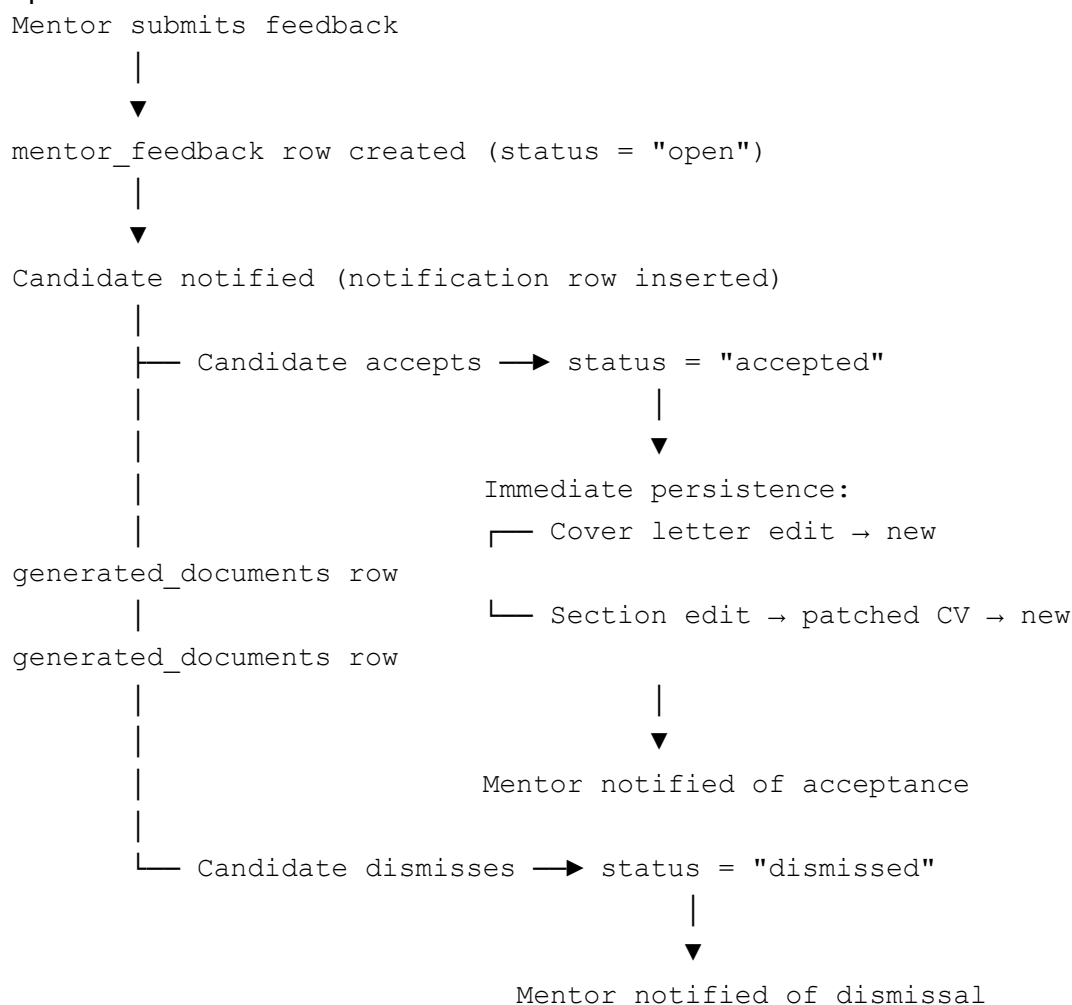
1. The candidate's baseline resume content.
2. Accepted bullet rewrites.
3. Mentor-accepted section rewrites.
4. Mentor-accepted cover letter edits.

The Express API applies these layers at generation time, ensuring that the latest accepted mentor edits are always reflected in every newly generated document without requiring candidates to manually re-trigger generation.

Documents are exported as DOCX (via python-docx) or PDF (via WeasyPrint). Generated content is stored in the `generated_documents` table as an append-only log — each generation produces a new row, so the full document history is preserved.

7.15 Mentor Feedback Lifecycle (`server/routes/mentor.js`, `server/routes/candidate.js`)

The mentor feedback system persists structured feedback items through a well-defined state machine involving mentor submission, candidate acceptance/dismissal, and notification updates.



The immediate persistence on acceptance is a deliberate design decision. Persisting the accepted content the moment acceptance is confirmed means the candidate's document view always reflects the latest agreed state without any additional action.

7.16 Notification System (server/notifications.js)

An event-driven notification system keeps candidates and mentors informed without requiring them to actively poll for changes.

Events that trigger notifications include new analysis attempts, feedback submissions, and discussion posts.

Event	Notified party
New analysis attempt	All mentors in the candidate's review sessions
Mentor submits feedback	Candidate
Candidate accepts feedback	Mentor
Candidate dismisses feedback	Mentor
New discussion annotation posted	The other party in the thread

Notifications are stored as rows in the database. Unread items are surfaced as highlighted rows in history tables, helping users track recent collaborative activity.

7.17 Hallucination Mitigation

One of the most significant challenges when applying Large Language Models to resume writing is hallucination. Language models frequently produce convincing yet fabricated achievements, percentages, or productivity improvements despite no supporting evidence existing within the original resume. For a document intended to represent a person's professional experience, such hallucinations undermine credibility.

Several design decisions were incorporated to reduce the likelihood of hallucinated output:

- Every rewrite is grounded using retrieved resume-writing frameworks rather than unconstrained prompting.
- Prompts explicitly instruct the model not to invent numerical metrics or unsupported accomplishments.
- Where measurable outcomes are desirable but unavailable, the application instructs the model to produce placeholders instead of fabricated values.

- Users retain complete editorial control — no rewrite is automatically applied to the final document. Every recommendation is presented individually for review, acceptance, or rejection.
- The optional critic LLM pass provides an additional independent validation layer before any rewrite reaches the user.

Collectively, these measures substantially reduce — but do not entirely eliminate — the possibility of fabricated resume content. The human-in-the-loop review step remains the final safeguard, reflecting the project's central philosophy: AI should support professional writing, not replace human judgement.

8. User Interface & User Experience

Although the analytical capabilities of RAGsToRiches form the technical core of the project, equal emphasis was placed on designing an intuitive, transparent, and engaging user experience. Throughout development, the frontend underwent several major redesigns as new features were introduced and opportunities emerged to simplify navigation, improve visual hierarchy, and better communicate AI-generated recommendations.

Rather than functioning as a conventional chatbot or automated document editor, the interface was designed around a guided review workflow. Users progress naturally from uploading their resume, through structured analysis, to interactive refinement, with each stage exposing only the information necessary for the current task. This progressive disclosure prevents users from becoming overwhelmed while still providing complete visibility into the reasoning behind every recommendation.

Unlike many commercial AI resume tools that immediately present a fully rewritten document, RAGsToRiches deliberately keeps users involved throughout the revision process. Every recommendation is surfaced explicitly, accompanied by supporting explanations, and must be consciously accepted before becoming part of the final output.

The overarching design philosophy can therefore be summarised in a single principle:

The user should remain the final decision-maker.

The interface exists not simply to present AI-generated content, but to help users understand, evaluate, and confidently improve their own resumes.

8.1 Interface Design Philosophy

Several principles guided every redesign of the frontend.

Simplicity

Resume optimisation is already a cognitively demanding task. The interface therefore minimises unnecessary controls, clutter, and visual distractions. Every screen focuses on a single primary objective, allowing users to concentrate on improving their resume rather than learning how to operate the application.

Progressive Disclosure

Rather than displaying every analysis result simultaneously, information is revealed incrementally throughout the workflow.

1. Upload Resume
2. Parse Resume
3. Analyse Resume
4. Review Suggestions
5. Accept / Reject Rewrites
6. Generate CV & Cover Letter
7. Final Manual Editing
8. Export (Markdown / DOCX / PDF)

Breaking the workflow into clearly defined stages reduces cognitive overload while allowing users to focus on one decision at a time.

Human-Centred AI

The application deliberately avoids presenting AI-generated recommendations as unquestionable truths.

Instead, every rewrite is accompanied by contextual explanations and requires explicit user approval before being incorporated into the final resume. This reinforces the project's broader philosophy that artificial intelligence should enhance human decision-making rather than replace it.

8.2 Interface Evolution

The user interface evolved substantially throughout development.

Early prototypes prioritised functionality, exposing numerous controls simultaneously and requiring users to manually navigate between loosely connected analysis stages. As additional capabilities; including resume scoring, Retrieval-Augmented Generation, ATS analysis, score tracking, and interactive rewrite acceptance - were introduced, the interface gradually became more complex.

Rather than continuing to add functionality onto the original design, the frontend was comprehensively redesigned around a task-oriented workflow.

The final interface introduces a cleaner visual hierarchy, improved spacing, consistent typography, and a more intuitive navigation flow that naturally guides users from resume upload through to document generation. Related functionality is grouped together, repetitive

interactions have been eliminated wherever possible, and unnecessary visual clutter has been removed to ensure that the user's attention remains focused on the analysis itself.

The result is an interface that feels significantly more approachable while simultaneously supporting a considerably richer feature set than earlier iterations.

8.3 React SPA Frontend

The frontend is implemented as a Single Page Application (SPA) using React and Vite. While the project originally used Streamlit for rapid prototyping, the architectural requirements quickly outgrew its capabilities. Delivering a highly interactive review experience, real-time notification badges, complex multi-panel mentor workspaces, and robust multi-user sessions required a dedicated presentation layer.

Moving to React offered several advantages:

- **Explicit State Management:** `useState` and `useEffect` hooks provide granular control over the UI, preventing the expensive full-page reruns common in early Streamlit prototypes.
- **Rich Interactivity:** The UI can support seamless navigation, collapsible diff viewers, interactive highlights, and asynchronous polling without interrupting the user's workflow.
- **Separation of Concerns:** The React frontend communicates strictly through the Express REST API, meaning frontend presentation and backend analysis can evolve independently.
- **Modern Styling:** Using Vanilla CSS with a consistent design token set enables glassmorphism effects, micro-animations, and responsive dark/light mode toggles.

The frontend functions purely as an orchestration and presentation layer. Its responsibilities include:

- Uploading resumes and job descriptions.
- Displaying parsed resume content alongside a sticky PDF viewer.
- Rendering analysis results and rewrite suggestions.
- Managing user interactions (accept/dismiss).
- Managing mentor workspaces and notification inboxes.
- Displaying generated final documents for manual editing.

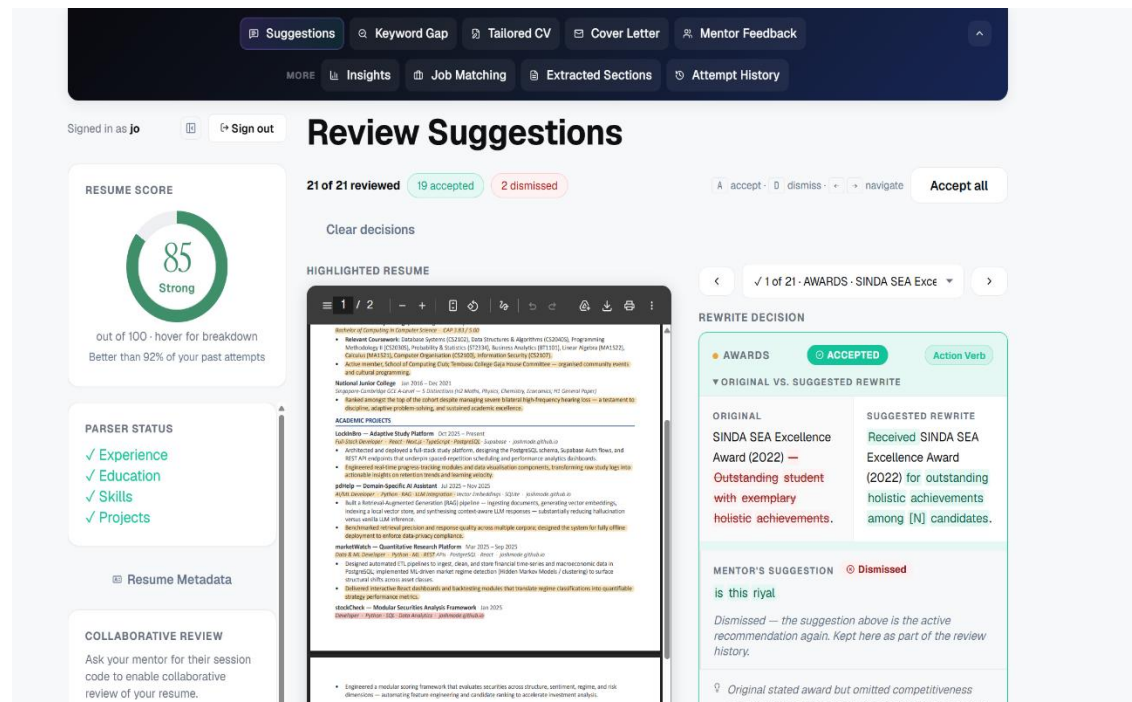
Importantly, no language model inference or complex parsing occurs within the browser itself. All heavy computational analysis remains encapsulated within the Python engine and orchestrated by the Express backend.

Importantly, no language model inference or complex parsing occurs within the browser itself. All heavy computational analysis remains encapsulated within the Python engine and orchestrated by the Express backend.

8.3.1 Visual Interface Design

Beyond workflow design, considerable effort was invested into creating a modern, uncluttered interface that remains approachable even for first-time users. The design system uses CSS variable tokens for colours, radii, spacing, and shadows to ensure a cohesive look.

Rather than exposing every analytical output simultaneously, the interface organises related information into clearly separated cards. This allows users to focus on a single task at a time while maintaining awareness of their overall progress through the analysis pipeline.



8.3.2 Structured Resume Viewer

Rather than displaying extracted text in a plain text box, the application provides a dual-panel view. The left panel shows parsed resume content organised into collapsible sections (Education, Experience, Skills, etc.), while the right panel features a sticky PDF viewer showing the original document. This allows users to quickly verify that document parsing has succeeded and cross-reference suggestions against their original document layout without constantly switching tabs.

8.3.3 Transparent Resume Scoring

Instead of presenting only a single numerical score, the application allows users to inspect how the overall score was calculated.

The Insights dashboard breaks down the score into individual components, including measurable impact, action verb strength, structural clarity, keyword coverage, and identified warnings. By exposing the underlying scoring methodology, the interface reinforces the project's philosophy that AI recommendations should remain explainable rather than opaque.

8.3.4 Interactive Rewrite Cards

One of the most distinctive interface components is the rewrite review card. Each recommendation presents:

- The original bullet point
- The AI-suggested rewrite
- The framework used (such as Google XYZ or STAR Method)
- A concise explanation describing why the recommendation was generated
- A severity badge (red / yellow / green) indicating the weakness of the original bullet

This side-by-side presentation allows users to compare improvements directly while understanding the reasoning behind every modification. Rather than functioning as an automated editing tool, the interface encourages active review and informed decision-making.

- A concise explanation describing why the recommendation was generated
- A severity badge (red / yellow / green) indicating the weakness of the original bullet

This side-by-side presentation allows users to compare improvements directly while understanding the reasoning behind every modification. Rather than functioning as an automated editing tool, the interface encourages active review and informed decision-making.

8.4 Interactive Rewrite Review

A central component of the user experience is the **interactive rewrite review interface**. RAGsToRiches deliberately avoids autonomous document editing; instead, it surfaces every AI recommendation as an individual decision point within a human-in-the-loop workflow. For each proposed improvement, candidates may:

- Formally **accept** the suggestion
- Explicitly **dismiss** the suggestion

- Defer the decision for subsequent evaluation

This granular interaction model prioritises **transparency**, ensuring that candidates remain fully conscious of every modification. By presenting original and rewritten formulations side-by-side with accompanying explanations, the interface functions as a pedagogical tool that helps users understand the principles of effective technical writing. Critically, this approval mechanism serves as a final safeguard against hallucinations, preventing fabricated metrics or unsupported claims from entering the final document.

Once the review cycle concludes, the platform synthesises the approved content to generate tailored application materials via the Express backend (`server/routes/generation.js`). This automated pipeline integrates accepted bullet rewrites and mentor-driven section overrides to ensure the exported documents reflect the latest synchronised state.

Users may further refine these documents within an *in-browser editor* before exporting to Word or PDF formats. This final stage allows for stylistic polishing and manual personalisation, reinforcing the project's commitment to empowering human judgement over opaque agentic automation.

```
□sys_prompt = (
    "You are an expert CV writer. Generate a professional, ATS-
friendly CV from the candidate's resume. "
    "Use accepted rewrite decisions as the candidate's preferred
wording. "
    "For dismissed rewrite suggestions, use the ORIGINAL text
verbatim – do not apply any part of the suggested rewrite. "
    "Never invent employers, dates, credentials, projects,
metrics, or skills not present in the original resume. "
    "Every bullet point must be grounded in information explicitly
provided in the candidate's resume. "
    "Write in a grounded, professional tone. Avoid corporate
fluff. "
    f"\n\nBANNED WORDS (never use these): {_BANNED_WORDS}\n"
    "\n\nFORMATTING RULES – follow these exactly:\n"
    "1. Output ONLY the final CV in Markdown format, nothing
else.\n"
    "2. The CV MUST fit within 2 pages (~600–750 words). Be
concise.\n"
    "3. Start with the candidate's name as # Name.\n"
    "4. Contact details on a single line separated by ` | `.\n"
    "5. Each section: ## SECTION NAME in ALL CAPS, followed by `--
-`.\n"
    "6. Job/project titles as ### or **_Title_**, dates on the
same line after ` | `.\n"
    "7. All bullets start with a strong ATS action verb in past
tense.\n"
```

```
"8. Limit 2-4 bullets per role. Cut ruthlessly.\n"
"9. Sections order: Contact, Summary, Experience, Education,\nSkills, Projects, Certifications.\n"
"10. No preamble, no explanation – CV text only."
)
```

□

This final editing stage allows users to refine wording, personalise sections, make stylistic adjustments, or introduce additional information before downloading the completed documents. By providing a final opportunity for manual revision, the application reinforces the principle that AI should assist rather than replace human judgement, ensuring users retain complete ownership over the final application materials.

The interface therefore functions less as an automated editing tool and more as an intelligent resume coach, guiding users through a collaborative review process from initial analysis to the generation of application-ready documents.

The screenshot displays a web application interface for resume analysis and editing. The top navigation bar includes a back arrow, 'All candidates', a user profile icon 'jo', and a 'More' dropdown. Below this, there are tabs for 'Resume Analysis' (active) and 'Cover Letters'. The main content area is divided into several sections:

- Analysis History:** Shows 'ANALYSIS #21 • 85/100 • 23 JUL 2026, 17:18' with buttons for 'Original PDF' and 'Rewritten Preview'.
- COMPARE REVISIONS:** Features dropdowns for 'Before...' and 'After...' and a 'Compare' button.
- EXTRACTED SECTIONS:** Includes an 'AWARDS' section with a list of achievements (SINDA SEA Excellence Award, MOE Edusave Bursary Award, American Mathematics Olympiad) and a 'YOUR SUGGESTION' box showing a dismissed suggestion 'bfff' with a 'Revise & resend' button.
- Resume Preview:** A large window showing the 'JOSHUA ANDREW' resume. The resume is structured with sections: PROFILE, TECHNICAL SKILLS, EDUCATION, and ACADEMIC PROJECTS. The 'TECHNICAL SKILLS' section lists languages, frameworks, and databases. The 'EDUCATION' section details the user's degree from the National University of Singapore and a diploma from Singapore Cambridge. The 'ACADEMIC PROJECTS' section describes the 'LockinBro' adaptive study platform and the 'pdHelp' domain-specific AI assistant.

The interface uses a clean, modern design with a light blue and white color scheme. The resume preview window is styled to look like a document with a dark header and clear section breaks.

8.5 Session State Management

Delivering a seamless review experience required careful management of application state.

In the old Streamlit architecture, state was volatile and lost when the script stopped running unless explicitly cached in memory. In the modern three-tier architecture, application state is managed cleanly across two layers:

1. **Client-Side UI State (React):** Short-lived state such as currently open accordions, typed but unsaved edits, modal visibility, and dark/light mode preferences are handled entirely within the browser using React's `useState` and `useEffect` hooks.
2. **Persistent Application State (SQLite):** Core business data — including uploaded resumes, generated AI suggestions, accepted rewrites, mentor feedback, and user accounts — is immediately persisted to the SQLite database by the Express API.

This strict separation means a user can safely close their browser halfway through reviewing a 20-bullet resume. When they log back in, the React frontend fetches the attempt from the database, and they resume exactly where they left off without needing to rerun any expensive LLM analysis. Furthermore, when a candidate accepts a mentor's cover letter edit, it is instantly written to the `generated_documents` table, meaning the candidate's view updates immediately without requiring a manual regeneration step.

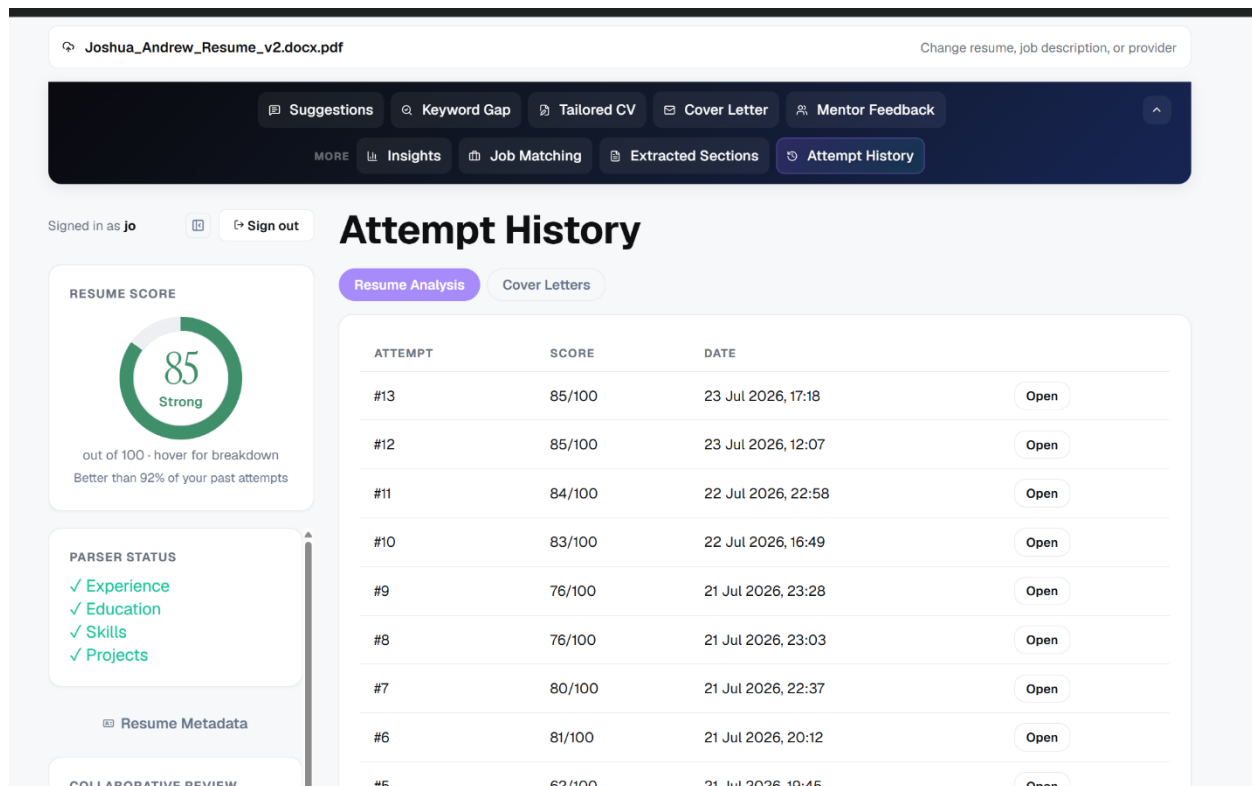
8.6 Progress Tracking

Resume writing is inherently iterative. Rather than treating every upload as an isolated analysis, RAGsToRiches records resume score history across successive evaluations, allowing users to observe measurable improvements over time.

The application features a dedicated **Insights** dashboard and **Attempt History** view. These interfaces provide:

- Chronological lists of all past analyses, labeled by target company.
- Keyword gap visualisations comparing historical resumes against their respective job descriptions.
- Score trend charts highlighting progression in overall quality.
- Section heatmaps identifying which parts of the resume consistently score poorly.

Tracking historical performance provides objective evidence of improvement, motivates iterative refinement, and helps candidates identify which revisions had the greatest impact on their scores.



8.7 Explainability by Design

Explainability represents one of the defining characteristics of the entire user experience.

Many AI writing assistants expose only their final output, leaving users unable to determine whether suggestions are grounded in recognised writing principles or generated solely through model intuition.

RAGsToRiches instead attempts to expose the reasoning process wherever practical.

Examples include:

- Explaining why individual bullets received lower scores.
- Highlighting missing ATS-related concepts.
- Presenting the retrieved resume-writing framework that informed each recommendation.
- Displaying original and rewritten bullets side-by-side.
- Allowing users to review every recommendation individually before accepting it.

This emphasis on explanation transforms the application from a simple AI text generator into a genuine educational tool.

Rather than merely producing stronger resumes, the system helps users develop a deeper understanding of effective professional writing, enabling them to apply these principles independently in future resumes.

9. Repository Structure

The repository follows a modular architecture that separates user interface components, language model orchestration, semantic retrieval, and document processing into distinct modules.

This organisation reflects the processing pipeline itself and simplifies maintenance by ensuring each component has a clearly defined responsibility.

RAGsToRiches/

```
|
|
| └─ client/                # React frontend (Vite)
|
|   └─ src/
|       └─ App.jsx          # Main SPA (~170KB, all views and components)
|       └─ index.css        # Design system and all component styles
|       └─ main.jsx         # React entry point
|       └─ api/
|           └─ client.js     # Axios client with auto-retry on 429
|           └─ context/
|               └─ AuthContext.jsx # JWT auth context
|           └─ package.json
|
|
| └─ server/                # Express REST API (ESM, Node.js)
|
|   └─ index.js             # Server entry, middleware, route mounting
|
|   └─ db.js               # SQLite schema + all database helpers
|
|   └─ notifications.js     # notify() / notifyMany() utilities
|
|   └─ userKeys.js          # Per-user encrypted API key management
```

- | | — engineClient.js # Internal HTTP client to Python engine
- | | — access.js # Role-based access helpers
- | | — middleware/
- | | | — rateLimit.js # Three-bucket Express rate limiter
- | | — routes/
- | | — auth.js # Register, login, JWT
- | | — analysis.js # Run, poll, history, keyword gap, quick CL
- | | — generation.js # Tailored CV, cover letter, mentor overrides
- | | — mentor.js # Mentor workspace, feedback lifecycle
- | | — annotations.js # Discussion threads on feedback items
- | | — notifications.js # Mark-read, summary badge counts
- | | — feedback.js # Candidate feedback route helpers
- | | — jd.js # Saved job descriptions
- | | — pdf.js # Highlighted PDF serving
- |
- | — engine_api.py # Flask API wrapping Python engine (internal only)
- | — analyser.py # Core analysis orchestration, batched rewriting
- | — parser.py # Resume parsing (PDF/DOCX/ODT/TXT/MD/LinkedIn)
- | — router.py # Multi-provider LLM router with retry + fallback
- | — vector_db.py # ChromaDB interface (RAG retrieval)
- | — mentor.py # Mentor-related Python utilities
- | — feedback.py # Feedback utility functions
- | — document_export.py # DOCX/PDF export generation
- | — pdf_highlight.py # Bullet annotation onto PDF bytes

— job_scraper.py	# JD keyword extraction + resume-JD comparison
— auth.py	# Python-side auth utilities
— db.py	# Python-side SQLite helpers (engine internal)
— chroma_db/	# Persistent ChromaDB vector store
— data/	# SQLite database (ragstoriches.db) + persistent state
— Dockerfile	# Multi-stage build (engine service)
— docker-compose.yml	# Base compose (Express API + engine + client)
— docker-compose.prod.yml	# Production overrides (security, env flags)
— docker-compose.https.yml	# Caddy TLS overlay (Let's Encrypt)
— .env.example	# All configurable environment variables
— requirements.txt	# Python dependencies (main engine)
— requirements-engine.txt	# Engine-specific deps (torch/easyocr optional)
— critic_benchmark.py	# Offline critic quality benchmark runner

This modular structure offers several practical advantages.

Separation of Concerns

Each module performs a single well-defined responsibility.

For example, the parser performs deterministic extraction only, while the router is concerned solely with communication with language models.

This reduces coupling between components and simplifies debugging.

Maintainability

Changes to one subsystem rarely require modifications elsewhere.

For instance, adding support for another language model generally involves extending the routing layer without altering parsing or frontend logic.

Similarly, improvements to the parser do not affect semantic retrieval or document generation.

Extensibility

Future features can be incorporated with minimal architectural disruption.

Potential extensions include:

- additional retrieval frameworks
- alternative embedding models
- new language model providers
- enhanced analytics dashboards
- support for additional document formats

Because responsibilities are already well isolated, these additions can typically be implemented without large-scale refactoring.

10. Deployment

RAGsToRiches is designed to be easily deployable both locally for development and on public infrastructure for production use. The entire application is containerised using Docker, ensuring that the complex Python dependencies (like PyMuPDF, EasyOCR, and ChromaDB) and the Node.js runtime environment behave identically across all environments.

10.1 Local Development (Docker)

To run the application locally, you only need Docker installed.

bash

1. Copy the environment template

`cp .env.example .env`

2. Generate secure random strings for the application secrets

You can generate these using: `openssl rand -hex 32`

Set `JWT_SECRET` (for user auth) and `KEY_ENCRYPTION_SECRET` (for BYOK API keys)

3. Set a default LLM provider key

e.g., Set `OPENROUTER_API_KEY` for the pooled free tier

4. Build and launch the containers

`docker compose up --build`

The application will be available at <http://localhost:3000>. In this configuration, the React SPA is served directly by the Express server without TLS.

10.2 Production Deployment (HTTPS, public internet)

For public deployment on a Virtual Private Server (VPS), RAGsToRiches includes a production-ready Docker Compose configuration that automatically handles TLS termination using Caddy.

bash

`cp .env.example .env`

1. Set all mandatory secrets (`JWT_SECRET`, `KEY_ENCRYPTION_SECRET`)

2. Configure your domain

Set `DOMAIN=resumes.example.com`

Set `FRONTEND_URL=https://resumes.example.com`

3. Launch with the Caddy TLS overlay

`docker compose \`

`-f docker-compose.yml \`

`-f docker-compose.prod.yml \`

```
-f docker-compose.https.yml \
up -d --build
```

How the Production Stack Works:

- **Caddy** acts as the edge router. It binds to ports 80 and 443, automatically provisions and renews SSL/TLS certificates via Let's Encrypt, and proxies all secure traffic to the Express API.
- **Express (Node.js)** receives the traffic, handles JWT authentication, serves the compiled React static files, and manages the SQLite database.
- **Python Engine** runs in a completely isolated container network. It binds only to the internal Docker network on port 5001 and is fundamentally unreachable from the public internet, protecting the expensive ML parsing and LLM orchestration endpoints from direct external access.

10.3 Persistent Storage Volumes

Because Docker containers are ephemeral, state must be explicitly persisted. The application relies on two persistent directories which are mapped as Docker volumes:

- **data/** — Contains the **ragstoriches.db** SQLite database file. This file holds the entire state of the application: user accounts, encrypted API keys, analysis histories, generated documents, mentor feedback, and notifications.
- **chroma_db/** — Contains the persistent vector embeddings for the resume-writing frameworks used in the RAG pipeline.

Important: You must back up the **data/** directory before any major redeployment or migration. Neither directory is included in source control.

10.4 Engine Memory & Concurrency Tuning

The Python analysis engine (**engine_api.py**) relies on heavy machine learning libraries (Torch, EasyOCR, Sentence-Transformers) which consume significant RAM.

To balance throughput with memory constraints on standard VPS hardware (e.g., 2GB-4GB RAM), the engine is executed via Gunicorn using a specific concurrency model: **1 worker process with 4 threads**.

- **Why 1 Worker?** Multiple processes would duplicate the heavy Sentence-Transformer embedding model in memory, leading to immediate Out-Of-Memory (OOM) crashes on smaller servers.
- **Why 4 Threads?** The engine's workload is primarily network I/O bound (waiting for external LLM API responses). Threads allow the engine to process up to four concurrent analyses efficiently without duplicating the baseline memory footprint.

10.4 Deployment Architecture Diagram

Internet

| :443 (HTTPS)



Caddy (TLS termination)

| :3000 (internal)



Express API (Node.js)

| — Serves React SPA (static build)

| — /api/* — REST endpoints

| | internal :5001



| Python Engine (Gunicorn, 1 worker + 4 threads)

| | — /analyse

| | — /gen-cv / /gen-cover-letter

| | — /compare-resume-jd

| | — /parse

| | — /highlight-pdf

| | — /export-docx

| — data/ (SQLite) + chroma_db/ (ChromaDB)

11. Design Rationale

Every significant architectural decision in RAGsToRiches was made to address a specific engineering challenge encountered during development. Rather than selecting technologies solely because they were popular, each component was chosen after considering maintainability, extensibility, deployment complexity, performance, and overall user experience.

This section discusses the rationale behind the major design decisions that shaped the final system.

11.1 Why Retrieval-Augmented Generation?

One of the earliest design questions concerned how resume-writing knowledge should be incorporated into the language model.

The simplest approach would have been to manually embed all resume-writing frameworks into every prompt.

While straightforward, this approach suffers from several disadvantages.

- Prompt length increases as additional frameworks are added, reducing available context windows.
- Irrelevant guidance is frequently included.
- Token usage becomes unnecessarily expensive.
- RAG forces LLM to retrieve AND use vectors, which may otherwise be ignored in a diluted prompt
- The model receives more context than required.
- There is little transparency regarding which framework influenced the final rewrite.

Retrieval-Augmented Generation addresses these limitations by separating knowledge storage from prompt construction.

Instead of permanently embedding resume-writing principles into prompts, the application retrieves only the framework fragments most relevant to the current resume bullet.

This produces shorter prompts, lower token usage, and considerably more targeted guidance.

By retrieving relevant resume-writing frameworks and injecting them directly into the model's context, RAG constrains the language model to generate recommendations that are explicitly informed by established industry standards instead of depending entirely on the model's internal knowledge or prompt interpretation.

Equally importantly, retrieval allows the application to expose which framework informed each recommendation, reinforcing the project's emphasis on explainability.

11.2 Why ChromaDB?

Several vector databases were evaluated during early design discussions.

The project ultimately adopted ChromaDB because it satisfied several practical requirements simultaneously.

First, it is lightweight enough to run locally without external infrastructure.

Second, it integrates naturally with Python, allowing the retrieval pipeline to remain relatively simple.

Third, its persistent storage model enables embeddings to survive application restarts without requiring repeated indexing.

For a project intended to support both cloud-hosted language models and completely offline execution, these characteristics made ChromaDB a particularly suitable choice.

11.3 Why a Private Python Engine?

Separating the Python analysis logic into an internal service (`engine_api.py`) provides several benefits:

- The Python-heavy dependencies (PyMuPDF, torch/easyocr, ChromaDB, sentence-transformers) are isolated in one container.
- The engine can be scaled, replaced, or updated independently of the API.
- The browser never has direct access to parsing or LLM endpoints, preventing abuse.
- The engine's memory footprint is predictable and manageable (1 worker + 4 threads, no process duplication of the embedding model).

11.4 Why SQLite?

SQLite via `better-sqlite3` was chosen over a client-server database (PostgreSQL, MySQL) because:

- Zero infrastructure — no separate database process or container.
- Synchronous API — matches the Express request/response model without callback complexity.
- Portable — the entire application state is a single file (`data/ragstoriches.db`), trivially backed up.
- Sufficient for the scale — resume analysis is read-mostly and traffic is bounded by LLM API rate limits, not database throughput.

11.4 Why Express (Node.js) for the API?

The Express API layer serves several purposes that would be awkward in a Python-only stack:

- **JWT authentication and account management** — Secure, stateless auth with bcrypt hashing and per-user encrypted API key storage.
- **SQLite via better-sqlite3** — Synchronous, embeddable, zero-infrastructure persistence without the complexity of PostgreSQL.
- **Async job management** — Analysis requests are queued and polled asynchronously, so long-running LLM calls don't block HTTP connections.
- **Rate limiting** — Three independent rate-limiter buckets (general, poll, LLM) prevent abuse without shared state across processes.
- **Internal-only engine** — The Python engine is bound to loopback and only reachable through Express, providing a security boundary.

11.5 Why a Modular Architecture?

The application deliberately avoids a monolithic implementation.

Instead, responsibilities are distributed across several independent modules.

Parser

↓

Analysis

↓

Retrieval

↓

Routing

↓

Presentation

This separation offers several engineering benefits.

- Individual components can be modified independently.
- Parser improvements do not affect retrieval.
- Language model providers can be replaced without changing frontend code.
- Frontend redesigns require little or no modification to semantic retrieval.

This loose coupling proved particularly valuable throughout development as the project evolved significantly from its original proposal.

11.6 Why Multiple LLM Providers?

Instead of presenting only a single numerical score, the application allows users to inspect how the overall score was calculated. The Insights dashboard breaks down the score into individual components, including measurable impact, action verb strength, structural clarity, keyword coverage, and identified warnings. By exposing the underlying scoring methodology, the interface reinforces the project's philosophy that AI recommendations should remain explainable rather than opaque.

However, during development several practical considerations emerged.

Different providers demonstrated different strengths.

Some produced stronger structured output. Others generated more natural resume rewrites. Some exhibited stricter rate limits. Others prioritised inference speed.

Furthermore, API pricing and model availability change rapidly.

Abstracting provider-specific logic behind a routing layer ensures that the application remains adaptable as the LLM ecosystem evolves.

Perhaps most importantly, supporting local Ollama execution allows privacy-conscious users to analyse sensitive resumes entirely offline.

11.7 Why Human-in-the-Loop Editing?

Many AI resume tools automatically rewrite the entire document before presenting it to the user.

Although convenient, this approach introduces two significant risks.

- 1) First, users often accept suggestions without carefully reviewing them.
- 2) Second, incorrect or hallucinated information can easily be incorporated into the final resume.

RAGsToRiches instead requires explicit approval for every suggested rewrite.

The workflow therefore becomes:

Original Bullet



AI Suggestion



User Review



Accept / Reject



Final Resume

Maintaining the user as the final decision-maker reflects one of the project's core principles:

Artificial intelligence should merely assist with professional writing; and not replace the professional judgement of its user.

11.8 Why a Mentor-Candidate Workflow?

The mentor/candidate system directly addresses the limitation that AI-only feedback, however well-grounded, cannot replicate the contextual judgement of an experienced human reviewer. By providing structured mentor feedback channels — with bullet edits, section rewrites, cover letter edits, discussion threads, and revision diffs — the platform enables the kind of iterative, coached improvement previously only available through expensive professional career services.

12. Engineering Challenges

Like most AI systems, RAGsToRiches evolved considerably throughout development.

Many implementation details changed as real-world testing exposed weaknesses that were not apparent during the initial design phase.

Rather than concealing these iterations, this section documents several of the most significant engineering challenges encountered during development.

12.1 Parsing Arbitrary Resume Layouts

Unlike academic assignments or structured datasets, resumes possess almost no standard formatting.

Applicants commonly use

- multiple columns
- icons
- decorative separators
- floating text
- tables
- custom typography
- unconventional section names

Simple PDF extraction frequently produces fragmented or incorrectly ordered text under these conditions.

Improving parser robustness therefore became one of the largest engineering efforts undertaken during the project.

Additional preprocessing stages - including section normalisation, duplicate removal, bullet reconstruction, and OCR fallback - were progressively introduced to improve extraction quality across diverse resume templates.

12.2 Hallucinated Resume Content

Large Language Models naturally attempt to generate convincing prose.

When asked to improve resume bullets, they frequently invent numerical metrics, percentages, or business outcomes that were never present in the original document.

Such hallucinations are particularly problematic in resumes because candidates are ultimately responsible for defending every claim during interviews.

Several mitigation strategies were therefore implemented.

- Retrieval-grounded prompting
- Explicit anti-hallucination instructions
- Placeholder values instead of fabricated statistics
- User approval for every rewrite

Collectively, these measures substantially reduce—but do not entirely eliminate—the possibility of fabricated resume content.

12.3 Supporting Multiple Providers

Although supporting multiple providers increases flexibility, it also introduces additional complexity.

Every provider exposes different

- authentication methods
- request formats
- response formats
- SDKs
- rate limits
- model identifiers

Without careful abstraction, these differences quickly spread throughout the codebase.

Implementing a dedicated routing layer isolated this complexity, allowing the remainder of the application to remain provider independent.

12.4 Maintaining Consistent State

The original Streamlit application maintained state through `st.session_state`, which was straightforward but limiting. React's `useState/useEffect` model gives precise control over re-renders but requires careful engineering of derived state, memoisation, and effect dependencies. The App component evolved to ~2800 lines with careful decomposition into logical functional sections while remaining a single cohesive module.

12.5 Balancing Explainability and Automation

Perhaps the most difficult design challenge involved determining how much autonomy should be given to the language model.

Fully automated rewriting produces impressive demonstrations but often leaves users unaware of why changes were made.

Conversely, exposing every intermediate stage risks overwhelming users with technical details.

The final design attempts to balance these competing objectives by automating repetitive analysis while requiring human approval for substantive content changes.

This balance aligns closely with the project's educational objective: helping users become stronger resume writers rather than simply generating stronger resumes.

12.5 Mentor Feedback Persistence and Consistency

Ensuring that accepted mentor edits immediately become canonical — without requiring the candidate to regenerate documents — required extending the `POST /feedback/:id/status` handler to:

1. Detect the type of feedback being accepted (cover letter edit vs. section edit).

2. Immediately insert a new `generated_documents` row with the accepted text (cover letter) or the patched CV (section edit).
 3. Return the persisted content to the frontend so the view refreshes without a separate API call.
-

12.6 Engine Memory Management

The Python engine previously imported `easyocr` and `torch` unconditionally at module load time. Combined with multiple Gunicorn workers, this caused memory pressure that triggered OOM kills in Docker. Two fixes resolved this: lazy import of OCR dependencies (only when genuinely needed by a scanned PDF), and switching from 2 worker processes to 1 worker + 4 threads (analysis is network-bound, not CPU-bound, so threads give equal concurrency without duplicating the embedding model across processes).

12.7 Analysis Caching and Smart Reuse

Running the same resume + JD + provider combination multiple times within a session would waste LLM API budget and add latency. The API now checks for an existing `analyses` row matching the same content hash, job description, provider, model, and critic setting within `ANALYSIS_CACHE_TTL_MINUTES` (default 60) and returns the cached result immediately if found.

13. Testing and Validation

Given the probabilistic nature of Large Language Models, traditional software testing alone is insufficient.

RAGsToRiches therefore combines deterministic testing for parser behaviour with scenario-based evaluation for language model output.

Testing focuses on several independent subsystems.

Parser Testing

The parser is evaluated using resumes exhibiting diverse layouts, including

- single-column resumes
- multi-column resumes
- scanned PDFs
- image-only resumes
- unconventional section headings

Testing verifies that sections are correctly identified and that bullet reconstruction remains reliable despite formatting variation.

Rewrite & Generation Evaluation

Rewrite suggestions, and subsequently generated CV/Cover letters were tested to ensure that

- suggestions are produced consistently
- navigation functions operate correctly
- accepted and rejected states persist
- generated content remains structurally valid and logical

Because language model output is inherently nondeterministic, evaluation emphasises behavioural correctness rather than exact textual equality.

End-to-end mentor/candidate workflow tests verify: feedback submission and notification delivery, acceptance persistence to `generated_documents`, discussion thread creation and retrieval, revision diff correctness across arbitrary attempt pairs, and mark-read propagation.

ATS Analysis

Keyword matching is validated using manually constructed resume and job-description pairs containing known overlaps and deliberately omitted terminology.

This ensures that semantic matching behaves consistently across representative examples.

Deployment Testing

Deployment testing verifies that the application executes correctly inside its Docker container, including

- dependency installation
- persistent ChromaDB storage
- score history persistence
- environment variable loading
- frontend accessibility

Containerisation ensures that identical behaviour can be reproduced across development and demonstration environments.

Critic Benchmark

A dedicated offline benchmark runner (`critic_benchmark.py`) evaluates critic quality by running paired critic-on/critic-off analyses against a fixture of known-good and known-bad rewrites, measuring hallucination catch rate and false-positive rejection rate

Overall, testing was designed not only to identify software defects but also to evaluate the reliability and consistency of AI-assisted resume analysis under realistic usage scenarios.

14. Current Features

The current implementation of RAGsToRiches represents the culmination of multiple iterations of architectural refinement and feature development. While several ideas proposed during the initial design phase remain candidates for future work, the system already delivers a complete end-to-end workflow spanning candidate resume analysis, interactive improvement, document generation, and structured mentor collaboration.

The following sections summarise the major capabilities currently implemented within the application.

14.1 Intelligent Document Parsing

The application accepts resumes across multiple formats (PDF, DOCX, ODT, TXT, Markdown, and LinkedIn ZIP exports) and converts them into structured, machine-readable representations suitable for downstream analysis.

Unlike naive text extraction utilities, the parser performs multiple stages of preprocessing to improve robustness across differing resume layouts. Current capabilities include:

- Format-specific text extraction (e.g., PyMuPDF for PDFs, `python-docx` for Word documents).
- Lazy-loaded OCR fallback (EasyOCR) for image-based resumes.
- Contact information extraction.
- Section detection and normalisation.
- Bullet point reconstruction.
- Duplicate removal.

These preprocessing stages significantly improve downstream language model performance by supplying cleaner, more structured input.

14.2 Resume Analysis & Agentic Validation

Following successful parsing, the resume undergoes a comprehensive evaluation. The analyser identifies opportunities for improvement across several dimensions, including structural quality, measurable impact, action verb usage, ATS compatibility, and overall writing effectiveness.

Rather than producing a single opaque judgement, the analyser generates detailed explanations that help users understand why particular sections require improvement. Additionally, an optional *critic* language model pass can be enabled to evaluate every suggested rewrite against strict hallucination and formatting guidelines, automatically repairing or rejecting problematic outputs before they reach the user.

14.3 Retrieval-Augmented Rewrite Generation

One of the defining capabilities of RAGsToRiches is its Retrieval-Augmented Generation (RAG) pipeline. Instead of asking the language model to rewrite bullets using unconstrained reasoning, the application first retrieves semantically relevant resume-writing frameworks (such as the Google XYZ or STAR Method) from a persistent ChromaDB vector store.

This retrieval-first workflow ensures that recommendations remain grounded in recognised professional writing principles rather than relying solely on the model's internal general knowledge.

14.4 Resume Scoring & Analytics

The application provides users with an overall assessment of resume quality alongside detailed explanations of identified weaknesses. Scores are broken down across multiple dimensions (impact, action verbs, clarity, ATS, quantification) and presented visually through the Insights dashboard, which includes section-level score heatmaps.

Rather than functioning as an arbitrary numerical ranking, the score serves as a diagnostic indicator. Because every score is accompanied by explanatory feedback, users are encouraged to understand the underlying reasoning instead of focusing exclusively on the numerical result.

14.5 Interactive Rewrite Review

Generated rewrite suggestions are intentionally presented within a structured review environment. For every recommendation, candidates can compare:

- the original resume bullet,
- the suggested rewrite,
- the retrieved framework used, and
- the explanation supporting the rewrite.

The user then decides whether to accept, dismiss, or leave the recommendation open. This review process preserves user ownership of the document while preventing unintended AI-generated modifications.

14.6 Automated CV and Cover Letter Generation

Following the review process, RAGsToRiches automatically generates a polished curriculum vitae and tailored cover letter using the candidate's approved content. Rather than treating document generation as an isolated task, the system synthesises information gathered throughout the analysis pipeline, including the original resume, accepted AI-generated rewrites, ATS keyword analysis, the target job description, and any mentor-approved section overrides.

A key design principle is that *only user-approved modifications* are incorporated. Suggestions that were rejected or dismissed are intentionally excluded, ensuring the generated CV accurately reflects the user's decisions.

The generated cover letter is similarly personalised. By jointly analysing the resume and job description, the language model constructs a role-specific cover letter that emphasises the candidate's strongest qualifications. If a mentor suggests an edit to a cover letter and the candidate accepts it, the system immediately persists this edit, eliminating the need to manually regenerate the document.

Documents are exported as professionally formatted Microsoft Word (DOCX) and Portable Document Format (PDF) files, allowing users to submit their applications immediately.

14.7 ATS Keyword Analysis & Job Matching

When a job description is supplied, the application performs semantic comparison between the uploaded resume and the target role. Rather than relying solely on literal keyword overlap, the analyser extracts key concepts from the JD and measures coverage within the resume.

The resulting feedback surfaces overall match percentages, identifies missing skills or critical gaps, auto-detects the company name, and offers specific tailoring tips to improve Applicant Tracking System compatibility while maintaining factual accuracy.

14.8 Persistent Progress Tracking & Multi-User Accounts

To encourage iterative resume improvement, the application supports secure, persistent user accounts governed by JWT authentication and SQLite persistence. Candidates can track their historical resume scores across successive analyses via a dedicated Attempt History view.

Tracking historical scores enables users to observe measurable improvements as revisions are incorporated. The application also provides visual score trend charts and tracks which accepted rewrites contributed to score improvements over time.

14.9 Multi-Provider & Local Language Model Support

Every provider exposes different capabilities, pricing, and privacy guarantees. RAGsToRiches implements a unified routing layer that abstracts these differences, supporting multiple LLM providers:

- OpenRouter (used for the pooled free tier)
- Google Gemini
- Anthropic Claude
- OpenAI
- Locally hosted Ollama models

The support for Ollama allows sensitive resumes to remain entirely on the user's own machine, ensuring absolute privacy while preserving the remainder of the application's functionality. Users can also securely provide their own API keys (Bring Your Own Key) using AES-256-GCM encryption.

14.10 Collaborative Mentor Workflow

Beyond automated AI feedback, RAGsToRiches introduces a comprehensive human-in-the-loop mentoring system. The platform allows career coaches and institutional staff to enrol candidates into review sessions.

Mentors have access to a dedicated workspace where they can view a candidate's complete attempt history, compare revisions using PR-style inline diffs, and inspect the original parsed PDF. Mentors can submit structured feedback (bullet edits, section rewrites, cover letter edits, and general comments) directly to the candidate's inbox. An event-driven notification system ensures both parties are alerted to new submissions, acceptances, and replies in per-feedback discussion threads.

14.11 User Feedback & Evaluation Channel

An integrated feedback mechanism allows users to voluntarily submit qualitative feedback regarding analysis quality, usability, and overall experience. Feedback is collected only with explicit user consent and is intended to support ongoing system evaluation and future improvements.

15. Future Work

Although the current implementation delivers a complete end-to-end resume analysis workflow, several planned improvements remain before the platform reaches its long-term vision. These future enhancements focus on improving reliability, deployment, usability, and extensibility while building upon the existing modular architecture.

15.1 Lightweight Local Critic

The current verification stage relies on the same family of frontier language models used for rewrite generation.

Future work will investigate replacing this critic with a lightweight locally hosted model dedicated solely to factual verification.

Potential benefits include:

- Lower API costs.
 - Significantly reduced verification latency and error mitigation.
 - Improved privacy.
 - Fully offline verification when paired with local generation models, possibly in a dual agent configuration.
-

15.2 Fine-Tuned Resume Model

The current implementation relies on general-purpose frontier language models guided through Retrieval-Augmented Generation and carefully engineered prompts.

A longer-term objective is to fine-tune a lightweight open-source model specifically for resume analysis and rewriting.

Rather than relying entirely on prompt engineering, the model would be trained using a synthetic dataset of resume bullets and high-quality rewrites generated from existing frontier models. Techniques such as LoRA or QLoRA would enable efficient fine-tuning while keeping inference lightweight enough for mass consumer hardware.

A specialised model has the potential to provide:

- Greater consistency across rewrites.
 - Lower hallucination rates.
 - Faster local inference.
 - Reduced operational costs.
 - Improved adherence to resume-writing frameworks, and perks of specificity.
-

15.3 Expanding the Retrieval Knowledge Base

The current semantic knowledge base focuses primarily on widely recognised resume-writing frameworks, including the Google XYZ Formula, the STAR Method, the Rule of Three, and ATS-oriented writing principles.

Future iterations aim to expand this corpus with additional domain-specific guidance, such as:

- Industry-specific resume conventions.
- Technical role-specific writing guidance.
- Leadership and management resume frameworks.
- Graduate school and research CV recommendations.
- Domain-specific action verb collections.

Expanding the knowledge base will allow the retrieval pipeline to provide increasingly specialised recommendations without requiring fundamental architectural changes or overt prompt-engineering.

15.4 Recruiter & Applicant Management System (Potential offshoot)

Although RAGsToRiches is currently designed primarily for individual job seekers, the underlying analysis pipeline could also support recruiter-facing workflows.

A longer-term direction is the development of an Applicant Management System (AMS) that leverages the existing analysis engine to assist recruiters in evaluating candidates more efficiently.

Potential capabilities include:

- Managing and organising multiple candidate resumes.
- Automatically analysing submitted resumes against job descriptions.
- Comparing candidate strengths, skill coverage, and ATS compatibility.
- Providing structured summaries to assist shortlisting decisions.
- Tracking candidate progress and review history throughout the recruitment process.

By extending the existing architecture beyond resume coaching, RAGsToRiches could evolve into a broader AI-assisted recruitment platform that supports both applicants and recruiters while retaining its emphasis on transparent, evidence-based analysis.

16. Lessons Learned

Developing RAGsToRiches reinforced several important software engineering principles.

The first was that deterministic preprocessing remains critically important even within AI-driven systems. Language models perform substantially better when supplied with clean, structured, high-quality inputs.

Consequently, considerable effort was invested in parser robustness, preprocessing, and normalisation before any model inference occurred.

Secondly, explainability proved just as important as raw model capability. Users are far more likely to trust recommendations when they understand the reasoning behind them. This insight motivated the decision to adopt Retrieval-Augmented Generation, expose supporting frameworks, and require explicit user approval for every rewrite.

Thirdly, modularity consistently paid dividends throughout development. As additional providers, retrieval mechanisms, and interface improvements were introduced, the decoupled architecture allowed individual components to evolve independently without destabilising the remainder of the system.

Finally, the project highlighted that building successful AI applications involves considerably more than just selecting a powerful language model.

Parser reliability, prompt construction, retrieval quality, response validation, user interaction design, deployment, and engineering discipline collectively contribute just as much to the overall quality of the system as the language model itself.

17. Contributors

Joshua Andrew

- System architecture
- Resume parser
- Analysis engine
- Retrieval pipeline
- LLM routing
- Prompt engineering
- Docker deployment
- Frontend integration
- Testing and optimisation

Huang Sijia

- Frontend development
- Resume Parser
- Analysis engine
- User interface design & implementation
- Feature integration
- System testing
- Documentation
- Evaluation

18. Acknowledgements

This project was developed as part of the **National University of Singapore (NUS) Orbital Programme 2026**, under the **Apollo 11** level of achievement.

We would like to thank our Orbital mentors, teaching team, and peers for their guidance and feedback throughout the development process.

We also acknowledge the open-source projects that made this work possible, including:

- Streamlit
- ChromaDB
- PyMuPDF
- Sentence Transformers
- Hugging Face
- EasyOCR
- pdf2image
- Google Gemini
- Anthropic Claude
- OpenAI
- Ollama

Without these libraries and platforms, the development of an accessible, explainable AI-powered resume analysis system would not have been possible.

19. Final Remarks

RAGsToRiches began with a simple question:

Can AI effectively improve resumes without asking users to blindly trust it?

The answer ultimately shaped every architectural decision made throughout the project.

Instead of treating language models as unquestionable authorities, the application combines deterministic preprocessing, semantic retrieval, structured prompt construction, and transparent user interaction into a system that emphasises collaboration over automation.

Every recommendation is grounded in retrieved resume-writing frameworks.

Every rewrite remains subject to human approval.

Every architectural component - from PDF parsing through semantic retrieval to multi-provider routing - was designed with modularity, transparency, and maintainability in mind.

Although the project will undoubtedly continue evolving, its central philosophy remains unchanged:

Artificial intelligence should empower, and not replace the judgement, experience, or voice of humans.